

**Билеты к экзамену**

Предмет: ООП

## Содержание

|    |                                                                                        |    |
|----|----------------------------------------------------------------------------------------|----|
| 1  | Эволюция языков программирования. Понятие класса, объекта                              | 3  |
| 2  | Абстрагирование – определение, пример, реализация на языке                             | 6  |
| 3  | Инкапсуляция – определение, пример, реализация на языке                                | 11 |
| 4  | Модульность, определение, пример использования                                         | 17 |
| 5  | Иерархия – определение, виды иерархий, примеры                                         | 20 |
| 6  | Наследование, виды наследования, примеры                                               | 26 |
| 7  | Реализация полиморфизма в языках программирования. Виртуальные методы                  | 32 |
| 8  | Абстрактные и виртуальные классы                                                       | 34 |
| 9  | Типизация. Статическое и динамическое связывание                                       | 37 |
| 10 | Статические члены класса. Дружественные члены класса                                   | 42 |
| 11 | Перегрузка операторов                                                                  | 45 |
| 12 | Реализация приведения типов в C++                                                      | 57 |
| 13 | Конструктор копирования                                                                | 59 |
| 14 | Использование паттернов. Пример использования паттерна Фабрика                         | 61 |
| 15 | Использование паттернов. Пример использования паттерна Одиночка                        | 69 |
| 16 | Использование паттернов. Пример использования паттерна Фабричный метод                 | 73 |
| 17 | Понятие жизненного цикла разработки программного обеспечения. Виды моделей ЖЦ          | 76 |
| 18 | Основные принципы создания хорошего кода                                               | 77 |
| 19 | Технологии COM, CORBA, .NET. Их идеология и использование. Автоматное программирование | 78 |
| 20 | Понятие тестирования. Основные типы тестирования                                       | 79 |

## Эволюция языков программирования. Понятие класса, объекта

Современные программные системы, эксплуатирующиеся в производстве, гораздо больше и сложнее тех, что были еще несколько лет назад. Эта тенденция продолжается на самом деле с начала 60-х годов, когда стало возможным говорить о промышленном программировании. Первые языки программирования были созданы для ухода от программирования в машинных кодах и на ассемблере и обладали существенным недостатком – разные данные не изолировались друг от друга и от различных подпрограмм. Поэтому, как только система начинала расти, лавинообразно накапливалось количество ошибок, конфликтов между данными и подпрограммами. Причем подпрограммы, по сути, были просто выделенными участками кода, которые также могли конфликтовать друг с другом. К числу таких языков относятся FORTRAN I, ALGOL-58, Flowmatic, IPL V.

С середины 60-х годов понимание этих проблем привело к появлению новых версий или новых языков, в которых большое внимание стало уделяться так называемым алгоритмическим абстракциям, и первой такой абстракцией стала процедурная абстракция. Теперь предполагалось, что задачу можно разделять на отдельные фрагменты по смыслу решаемых подзадач, использовать ранее созданные процедуры в качестве основы для проектирования новых программ, но на этом этапе продолжал отсутствовать контроль над взаимодействием данных и процедур. Примерами таких языков являются ALGOL-60, COBOL, Lisp.

К концу 60-х и началу 70-х годов в языки программирования проникла идея модульной структуры организации программы, когда код мог состоять из нескольких изолированных модулей со своими наборами изолированных друг от друга подпрограмм и наборами данных, появилась идеология передачи параметров в подпрограммы, а следовательно, появилась возможность разделять данные. Тем не менее многие недостатки еще продолжали сохраняться, особенно когда надо было разрабатывать очень большие проекты большим коллективом разработчиков. В это время появились такие известные и до сих пор широко используемые языки, как Pascal, C, Ada.

Можно говорить о том, что это время стало периодом расцвета процедурного (структурного) программирования. Процедурное программирование рассматривает программы как набор многократно используемых единиц кода (процедур), для которых определены входные и выходные параметры. Подпрограммы могут вызывать друг друга, даже если они находятся в различных модулях проекта.

Проблемой процедурного программирования является то, что повторное использование кода во многих случаях ограничено: если меняется тип данных, надо создавать новую подпрограмму, если одни и те же данные можно обрабатывать по-новому, то встает вопрос о том, как учесть или исключить возможность предыдущей реализации. При этом еще и необходимо помнить, как вообще можно обрабатывать те или иные данные в проекте. Иногда процедурный подход еще называют алгоритмическим, хотя вернее говорить было бы об алгоритмической декомпозиции задачи. При этом способе программирования во главу угла ставится алгоритм, который воздействует на некоторые данные. С одной стороны, это удобно, но если изменились данные, то насколько легко можно изменить алгоритм, особенно если он описывает взаимодействие многих

различных типов данных?

Следующим шагом в развитии теории разработки программного обеспечения стало появление идеи абстрактной структуры данных, вершиной которой стало понятие объекта. Гради Буч определяет **объект** как осязаемую реальность, проявляющую четко выделяемое поведение. С точки зрения восприятия человека, объектом может быть:

- осязаемый и (или) видимый предмет;
- нечто, воспринимаемое мышлением;
- нечто, на что направлена мысль или действие.

Принято множество всех объектов, обладающих одинаковой структурой и имеющих одинаковое поведение, называть **классом**. Говорят, что поведение класса реализуется через его методы (их еще называют членами класса), а структура данных – через атрибуты или характеристики.

Объектно-ориентированный подход основан на парадигме, согласно которой каждая программа работает с данными, которые описывают сущности (объекты или события) из реальной жизни. Например: бухгалтерские программные системы работают со счетами, товарами, складами, наличием, заказами на продажу и т.д., системы по продажам билетов – с пассажирами, рейсами, билетами, самолетами или поездами, маршрутами движения транспорта.

Во главу угла в этом случае ставится не алгоритмическая, а объектная декомпозиция задачи. Теперь выделяют объекты, которые взаимодействуют между собой для решения той или иной задачи. Такая идеология позволяет упростить построение сложных систем, так как теперь подсистему можно рассматривать как отдельный объект, у которого есть свой способ взаимодействия с другими объектами.

Разницу между алгоритмическим и объектным взглядом на разработку программного обеспечения можно увидеть на следующем примере.

**Пример.** На книжной полке лежат книги. Необходимо попросить другого человека принести одну из них.

**Алгоритмический вариант решения:** назвать книгу, точно описать человеку, где находится полка, где на ней книга и даже как её оттуда взять: выдвинуть первые три книги слева, положить их на стол, потом взять нужную книгу, положить её на стол, потом взять три книги и вернуть их на место. Мы точно описали, что надо делать, но, если изменятся условия (например, книга стоит дальше, человеку неудобно снимать три книги вместе, стол отодвинут далеко слишком или вообще убран, или нам вообще не важно, что будет с книгами, снятыми с полки), что будет с алгоритмом?

**Объектный вариант решения:** назвать книгу, попросить принести книгу. Вопрос о том, как достать книгу с полки, объект решает самостоятельно, в соответствии со своей моделью поведения и своими возможностями.

Надо отметить, что объектный подход к программированию никоим образом не исключает алгоритмический! Ведь свое поведение объекты реализуют в соответствии со своими алгоритмами поведения, и их всё равно придется проектировать и реализовывать. Преимущество объектного подхода заключается в том, что если у разработчика есть набор готовых объектов с известным поведением, то он может только определять их совместное поведение, не вдаваясь в реализацию для каждого отдельного объекта. При этом если реализация поведения объекта изменится, а результат останется прежним, то никто (кроме самого объекта) этого не заметит.

Первые объектные языки программирования, такие как Simula, использовали понятие объекта, но еще не являлись в полной мере объектноориентированными в нынешнем понимании этого термина. Чтобы говорить об объектноориентированном программировании (ООП) как о методологии программирования, основанной на представлении программы в виде совокупности объектов, каждый из которых является экземпляром определенного класса, а классы образуют иерархию наследования, необходимо было научиться рассматривать однотипные объекты как представителей одного класса (это увеличило возможность повторяемости кода) и ввести понятие иерархии сходных или сложных объектов.

Таким образом, для того чтобы язык программирования был объектноориентированным, он должен уметь работать с классами и объектами, а также реализовывать и использовать следующие основные объектноориентированные принципы:

- абстракция
- инкапсуляция
- наследование
- полиморфизм
- модульность

## Абстрагирование – определение, пример, реализация на языке

### Абстракция

Определения с лекции:

**Определение 1. Абстракция** - выделение существенных для решаемой задачи характеристик (атрибутов) и элементов поведения объектов.

При абстрагировании должен выполняться принцип "**барьера абстракции**" - выделить то, что нужно, но не больше и не меньше.

**Уровни абстракции** - на каждый объект можно посмотреть по-разному с точки зрения решаемой задачи.

**Определение 2. Понятие абстракции** необходимо для понимания того, как разработчик собирается описывать объект в программе. «Абстракция выделяет существенные характеристики некоторого объекта, отличающие его от всех других видов объектов, и, таким образом, четко определяет его концептуальные границы с точки зрения наблюдателя. Использование абстракции концентрирует внимание на внешних особенностях объекта и позволяет отделить самые существенные особенности поведения от несущественных».

Когда разработчик или аналитик начинает изучать предметную область, для которой надо разработать программу, основной проблемой становится понимание того, что надо знать об объекте для решения задачи, что является не нужным, а может быть даже и вредным. Можно сказать, что правильно построенная абстракция означает работу с чем-то, что мы знаем, как использовать, но не знаем, как это работает внутри, и не знаем об объекте ничего кроме того, что надо знать, чтобы с ним работать.

Например, совершенно не надо знать, как устроен внутри телевизор, чтобы включить его, интересует только вопрос о том, где находится кнопка «вкл/выкл» и как настроить каналы и громкость. Лишние знания об объекте можно сравнить с попыткой включить телевизор при снятой задней крышке – работать будет, но вопрос, как долго и насколько безопасно для окружающих.

Когда мы говорим об абстракции и инкапсуляции объекта, мы должны понимать, что речь на самом деле идет не об одном объекте, а о наборе одинаковых объектов. Что подразумевается под одинаковостью объектов в этом случае? Это совпадение их поведения как внешнего, так и внутреннего, и одинаковый набор характеристик.

**Определение 3.** Принято множество всех объектов, обладающих одинаковой структурой и имеющих одинаковое поведение, называть **классом**. Говорят, что поведение класса реализуется через его *методы* (их еще называют *членами класса*), а структура данных – через *атрибуты* или *характеристики*.

Когда разработчик проектирует абстракцию и определяет правила инкапсуляции, то он на самом деле разрабатывает класс, определяет его интерфейс (раздел `public`) и инкапсулирует его внутреннюю структуру и поведение. Затем в программе будут определены экземпляры класса или по-другому объекты, принадлежащие классу.

## Описание и использование класса, модификаторы доступа

Рассмотрим на примере использование принципов абстрагирования и инкапсуляции ООП.

**Пример.** Заказчик решил создать программу, моделирующую работу зоопарка. В начале было заявлено, что единственным видом животных в зоопарке являются медведи. Разработчик должен описать класс `Bear` и задать всем медведям какое-то поведение и характеристики. Заказчик сказал, что для него сейчас важны функции кормления медведя мёдом и движения медведя, также важен вес медведя, еще у каждого медведя есть имя. То есть поведение – умение есть, умение двигаться, а атрибуты – вес и имя. Тогда первый вариант кода будет выглядеть так:

```
class Bear {
public:
    double weight;
    string name;
    Bear() {}

    void IsEating() {
        cout << "Медведь " << " " << name << " ест мед";
    }

    void Go() {
        cout << "Медведь " << " " << name << " идет";
    }
};
```

Определение класса начинается со слова `class` и заканчивается точкой с запятой, содержит атрибут `name` – имя медведя, атрибут `weight` – вес медведя, и методы поведения `isEating()` и `Go()`, а также конструктор `Bear()`.

Каждый класс обязательно должен содержать хотя бы один конструктор, это требование является обязательным для всех языков программирования, в C++ имя конструктора совпадает с именем класса. Если у класса нет конструктора, то объект класса создать будет нельзя. В данном случае был использован так называемый пустой конструктор по умолчанию, который отвечает только за создание объекта в памяти, но не задает никаких значений для объекта. Другие варианты конструкторов будут рассмотрены ниже. В некоторых случаях в описании класса есть еще метод-деструктор, который отвечает за уничтожение объекта и очистку памяти, но так как для решения многих учебных задач можно положиться на обычные методы сборки мусора и очистки памяти, то деструкторы будут рассмотрены позднее.

Обычно принято разделять описание и реализацию класса, если это позволяют средства языка. В C++ такое возможно. Описание класса записывают в заголовочный файл "имякласса".h, а реализацию поведения – в файл "имякласса".cpp, к этому файлу директивой `#include` подключают .h-файл. Такое описание будет иметь вид:

```
// Bear.h
class Bear {
public:
```

```

    double weight;
    string name;
    Bear() {}
    void IsEating();
    void Go();
};

// Bear.cpp
#include <iostream>
#include "Bear.h"

void Bear::IsEating() {
    cout << "Медведь " << name << " ест мед\n";
}

void Bear::Go() {
    cout << "Медведь " << " " << name << " идет";
}

```

Имени метода в сpp-файле предшествует имя класса (**Bear** в данном случае) и оператор `::`, который означает, что метод `IsEating()` является членом класса `Bear`.

Когда классы небольшие по размеру и методы просты, часто весь код пишут в `.h`-файл, чтобы уменьшить количество файлов в проекте, но если затем описание класса начинает увеличиваться в размере, то файл становится плохо читаемым.

В программе обращение к классу можно осуществить следующим образом:

```

Bear bear;
bear.name = "Винни";
bear.weight = 10;
cout << "Медведь " << bear.name << " живет в зоопарке"
<< '\n';
bear.IsEating();
bear.Go();

```

В этом примере создан экземпляр класса **Bear** (или переменная типа `Bear`) с именем `bear`. Затем, так же, как и при работе со структурами, мы обратились к полю `name`, задав ему значение "Винни" и к полю `weight`. Потом были вызваны методы кормления медведя и движения.

В результате получим:

```

Медведь Винни живет в зоопарке
Медведь Винни ест мед
Медведь Винни идет

```

Надо отметить, что в данном коде есть существенный недостаток – имя медведя и его вес находятся в разделе описания `public`. С точки зрения грамотной работы с классами и объектами это недопустимо, так как это означает, что любой пользователь этого класса (сотрудник зоопарка) по ошибке или специально может изменить имя медведя или задать ему отрицательный вес, хотя в его обязанности входит только кормить зверя.

Это произошло из-за того, что при описании класса был использован только модификатор доступа `public` (если его не использовать, то по умолчанию будет установлен `private`, и работать с классом будет невозможно). Поэтому разработчик должен скрыть такие атрибуты от посторонних. Это будет сделано с помощью модификатора `private`. Но тогда возникнет проблема с тем, как узнать имя и вес животного. Если предположить, что имя животного не меняется, а задается при его рождении, то тогда имя должно определяться при создании объекта (т.е. при вызове конструктора), а вот вес может меняться, поэтому можно определить начальный вес животного и способы изменения веса.

Уже на этом примере становится видно, что изменение взгляда на инкапсуляцию повлекло за собой изменение абстракции объекта – придется добавлять новые методы к классу. Обычные правила установки типа доступа и определения интерфейса для работы с атрибутами объектов следующие:

1. Все атрибуты класса делаются закрытыми `private`

2. для доступа к атрибутам, значение которых не может меняться из обращающейся к классу программы, определяется метод-селектор (или геттер), который принято начинать со слова `get`, а вторая часть имени – это имя атрибута. Значения таких атрибутов должны задаваться либо при создании объекта в конструкторе, либо с помощью каких-либо скрытых `private` методов класса, описывающих внутреннее поведение объектов

3. для доступа к атрибутам, значение которых может меняться из обращающейся к классу программы, определяется также метод-селектор, но еще добавляется метод-модификатор (сеттер), который принято начинать со слова `set`, а вторая часть имени – это имя атрибута. При правильной разработке модификаторов желательно проверять допустимость вводимых значений атрибута. Начальное значение атрибута лучше задавать при создании класса, если это возможно.

Изменим описание класса следующим образом: можно только узнать имя медведя и вес медведя, вес медведя может меняться только в результате кормежки, начальный вес 10 кг, и при каждой кормежке корм может быть разного веса:

```
// Bear.h
class Bear {
private:
    double weight;
    string name;
    void setWeight(double we);
public:
    Bear(string n);
    void IsEating(double we);
    void Go();
    string getName();
    double getWeight();
};

// Bear.cpp
#include <iostream>
#include "Bear.h"

Bear::Bear(string n){
```

```

    name = n;
    weight = 10;
}

void Bear::IsEating(double we){
    if (we > 0) {
        setWeigth(we);
        cout << "Медведь " << name << " съел" << we << "кг меда\n";
    }
    else cout << "Медведю " << name << " не дали мед\n";
}

void Bear::setWeigth(double we){
    weight = weight + 0.1 * we; // 10% корма на прибавку к весу
}

void Bear::Go() {
    cout << "Медведь " << " " << name << " идет\n";
}
string Bear::getName() { return name; }
double Bear::getWeight() { return weight; }

```

Тогда предыдущий пример работы с классом примет вид:

```

Bear bear("Винни");
cout << "Медведь " << bear.getName() << " живет в зоопарке" << '\n';
bear.IsEating(3);
bear.Go();
cout << "Медведь " << bear.getName() << " весит " <<
bear.getWeight() << " кг\n";
bear.IsEating(-3); //корм отрицательного веса!

```

В результате получим:

```

Медведь Винни живет в зоопарке
Медведь Винни ест мед
Медведь Винни весит 13 кг
Медведь Винни идет
Медведю Винни не дали мед

```

Видно, что изменение уровня доступа изменило и способ работы с объектом, и если попытаться обратиться напрямую к полю `name`, например так:

```

bear.name = "Винни";

```

то будет получено сообщение об ошибке еще на стадии компиляции программы.

## Инкапсуляция – определение, пример, реализация на языке

### Инкапсуляция

**Определение 4. Инкапсуляция** – это процесс отделения друг от друга элементов объекта, определяющих его устройство и поведение; инкапсуляция служит для того, чтобы изолировать контрактные обязательства абстракции от их реализации.

Одной из функций инкапсуляции является еще и разграничивание абстракций между собой. Так покупатель видит только внешний интерфейс телевизора, для мастера по ремонту абстракция расширена – он видит платы или провода, установленные в телевизоре, но при этом не видит их внутреннее устройство.

В языках программирования инкапсуляция обычно реализуется с помощью модификаторов доступа `public`, `private`, `protected`. Модификатор `public` означает, что все компоненты объекта, описанные с ним, доступны для других пользователей, т.е. они могут обращаться к объекту через такие компоненты. Модификаторы `private`, `protected` скрывают соответствующие компоненты от посторонних.

Обычно все характеристики объекта описывают в `private` части, поведение объекта по отношению к внешнему миру определяется в `public` разделе, внутреннее поведение объекта – в `private` или `protected` частях описания.

Когда мы говорим об абстракции и инкапсуляции объекта, мы должны понимать, что речь на самом деле идет не об одном объекте, а о наборе одинаковых объектов. Что подразумевается под одинаковостью объектов в этом случае? Это совпадение их поведения как внешнего, так и внутреннего, и одинаковый набор характеристик. Принято множество всех объектов, обладающих одинаковой структурой и имеющих одинаковое поведение, называть классом. Говорят, что поведение класса реализуется через его методы (их еще называют членами класса), а структура данных – через атрибуты или характеристики.

Когда разработчик проектирует абстракцию и определяет правила инкапсуляции, то он на самом деле разрабатывает класс, определяет его интерфейс (раздел `public`) и инкапсулирует его внутреннюю структуру и поведение. Затем в программе будут определены экземпляры класса или по-другому объекты, принадлежащие классу.

### Отличие инкапсуляции от сокрытия

Инкапсуляция – одна из основных парадигм ООП, которая, по сути, представляет собой способность языка скрывать определённые участки кода в блоки, забирая тем самым возможность у пользователя (или любого другого стороннего субъекта) нарушать целостность данного кода, вмешиваться в реализацию и тд. Основной единицей инкапсуляции в C++ является класс. Инкапсуляция позволяет качественно структурировать код и помогает обезопасить его от многих возможных проблем и вопросов, касательно защиты данных и информации.

Итак, если инкапсуляция обеспечивает своего рода целостность объекта, то сокрытие – скрывает детали о процессе. Для "настройки" доступа к данным

в классе и к классу непосредственно используются так называемые модификаторы доступа. Так вот использование этих модификаторов доступа и есть сокрытие.

Термин «сокрытие», как правило, употребляется в контексте методов. Сокрытие метода представляет собой определение тела метода в классе-наследнике, сигнатура которого соответствует сигнатуре метода в базовом классе. Для техники сокрытия применяется ключевое слово **new**.

## Описание и использование класса, модификаторы доступа

Рассмотрим на примере использование принципов абстрагирования и инкапсуляции ООП.

**Пример.** Заказчик решил создать программу, моделирующую работу зоопарка. В начале было заявлено, что единственным видом животных в зоопарке являются медведи. Разработчик должен описать класс `Bear` и задать всем медведям какое-то поведение и характеристики. Заказчик сказал, что для него сейчас важны функции кормления медведя мёдом и движения медведя, также важен вес медведя, еще у каждого медведя есть имя. То есть поведение – умение есть, умение двигаться, а атрибуты – вес и имя. Тогда первый вариант кода будет выглядеть так:

```
class Bear {
public:
    double weight;
    string name;
    Bear() {}

    void IsEating() {
        cout << "Медведь " << " " << name << " ест мед";
    }

    void Go() {
        cout << "Медведь " << " " << name << " идет";
    }
};
```

Определение класса начинается со слова `class` и заканчивается точкой с запятой, содержит атрибут `name` – имя медведя, атрибут `weight` – вес медведя, и методы поведения `isEating()` и `Go()`, а также конструктор `Bear()`.

Каждый класс обязательно должен содержать хотя бы один конструктор, это требование является обязательным для всех языков программирования, в C++ имя конструктора совпадает с именем класса. Если у класса нет конструктора, то объект класса создать будет нельзя. В данном случае был использован так называемый пустой конструктор по умолчанию, который отвечает только за создание объекта в памяти, но не задает никаких значений для объекта. Другие варианты конструкторов будут рассмотрены ниже. В некоторых случаях в описании класса есть еще метод-деструктор, который отвечает за уничтожение объекта и очистку памяти, но так как для решения многих учебных задач можно положиться на обычные методы сборки мусора и очистки памяти, то деструкторы будут рассмотрены позднее.

Обычно принято разделять описание и реализацию класса, если это позволяют средства языка. В C++ такое возможно. Описание класса записывают в заголовочный файл "имякласса".h, а реализацию поведения – в файл "имякласса".cpp, к этому файлу директивой `#include` подключают .h-файл. Такое описание будет иметь вид:

```
// Bear.h
class Bear {
public:
    double weight;
    string name;
    Bear() {}
    void IsEating();
    void Go();
};

// Bear.cpp
#include <iostream>
#include "Bear.h"

void Bear::IsEating() {
    cout << "Медведь " << name << " ест мед\n";
}

void Bear::Go() {
    cout << "Медведь " << " " << name << " идет";
}
```

Имени метода в cpp-файле предшествует имя класса (`Bear` в данном случае) и оператор `::`, который означает, что метод `IsEating()` является членом класса `Bear`.

Когда классы небольшие по размеру и методы просты, часто весь код пишут в .h-файл, чтобы уменьшить количество файлов в проекте, но если затем описание класса начинает увеличиваться в размере, то файл становится плохо читаемым.

В программе обращение к классу можно осуществить следующим образом:

```
Bear bear;
bear.name = "Винни";
bear.weight = 10;
cout << "Медведь " << bear.name << " живет в зоопарке"
<< '\n';
bear.IsEating();
bear.Go();
```

В этом примере создан экземпляр класса `Bear` (или переменная типа `Bear`) с именем `bear`. Затем, так же, как и при работе со структурами, мы обратились к полю `name`, задав ему значение "Винни" и к полю `weight`. Потом были вызваны методы кормления медведя и движения.

В результате получим:

```

Медведь Винни живет в зоопарке
Медведь Винни ест мед
Медведь Винни идет

```

Надо отметить, что в данном коде есть существенный недостаток – имя медведя и его вес находятся в разделе описания `public`. С точки зрения грамотной работы с классами и объектами это недопустимо, так как это означает, что любой пользователь этого класса (сотрудник зоопарка) по ошибке или специально может изменить имя медведя или задать ему отрицательный вес, хотя в его обязанности входит только кормить зверя.

Это произошло из-за того, что при описании класса был использован только модификатор доступа `public` (если его не использовать, то по умолчанию будет установлен `private`, и работать с классом будет невозможно). Поэтому разработчик должен скрыть такие атрибуты от посторонних. Это будет сделано с помощью модификатора `private`. Но тогда возникнет проблема с тем, как узнать имя и вес животного. Если предположить, что имя животного не меняется, а задается при его рождении, то тогда имя должно определяться при создании объекта (т.е. при вызове конструктора), а вот вес может меняться, поэтому можно определить начальный вес животного и способы изменения веса.

Уже на этом примере становится видно, что изменение взгляда на инкапсуляцию повлекло за собой изменение абстракции объекта – придется добавлять новые методы к классу. Обычные правила установки типа доступа и определения интерфейса для работы с атрибутами объектов следующие:

1. Все атрибуты класса делаются закрытыми `private`
2. для доступа к атрибутам, значение которых не может меняться из обращающейся к классу программы, определяется метод-селектор (или геттер), который принято начинать со слова `get`, а вторая часть имени – это имя атрибута. Значения таких атрибутов должны задаваться либо при создании объекта в конструкторе, либо с помощью каких-либо скрытых `private` методов класса, описывающих внутреннее поведение объектов
3. для доступа к атрибутам, значение которых может меняться из обращающейся к классу программы, определяется также метод-селектор, но еще добавляется метод-модификатор (сеттер), который принято начинать со слова `set`, а вторая часть имени – это имя атрибута. При правильной разработке модификаторов желательно проверять допустимость вводимых значений атрибута. Начальное значение атрибута лучше задавать при создании класса, если это возможно.

Изменим описание класса следующим образом: можно только узнать имя медведя и вес медведя, вес медведя может меняться только в результате кормежки, начальный вес 10 кг, и при каждой кормежке корм может быть разного веса:

```

// Bear.h
class Bear {
private:
    double weight;
    string name;
    void setWeight(double we);
public:
    Bear(string n);
    void IsEating(double we);

```

```

    void Go();
    string getName();
    double getWeight();
};

// Bear.cpp
#include <iostream>
#include "Bear.h"

Bear::Bear(string n){
    name = n;
    weight = 10;
}

void Bear::IsEating(double we){
    if (we > 0) {
        setWeigth(we);
        cout << "Медведь " << name << " съел" << we << "кг меда\n";
    }
    else cout << "Медведю " << name << " не дали мед\n";
}

void Bear::setWeigth(double we){
    weight = weight + 0.1 * we; // 10% корма на прибавку к весу
}

void Bear::Go() {
    cout << "Медведь " << " " << name << " идет\n";
}

string Bear::getName() { return name; }
double Bear::getWeight() { return weight; }

```

Тогда предыдущий пример работы с классом примет вид:

```

Bear bear("Винни");
cout << "Медведь " << bear.getName() << " живет в зоопарке" << '\n';
bear.IsEating(3);
bear.Go();
cout << "Медведь " << bear.getName() << " весит " <<
bear.getWeight() << " кг\n";
bear.IsEating(-3); //корм отрицательного веса!

```

В результате получим:

```

Медведь Винни живет в зоопарке
Медведь Винни ест мед
Медведь Винни весит 13 кг
Медведь Винни идет
Медведю Винни не дали мед

```

Видно, что изменение уровня доступа изменило и способ работы с объектом, и если попытаться обратиться напрямую к полю `name`, например так:

```
bear.name = "Винни";
```

то будет получено сообщение об ошибке еще на стадии компиляции программы.

## Модульность, определение, пример использования

### Модульность

Понятие модульности. По мнению Майерса "Разделение программы на модули до некоторой степени позволяет уменьшить ее сложность... Однако гораздо важнее тот факт, что внутри модульной программы создаются множества хорошо определенных и документированных интерфейсов. Эти интерфейсы неоценимы для исчерпывающего понимания программы в целом". В некоторых языках программирования, например в Smalltalk, модулей нет, и классы составляют единственную физическую основу декомпозиции. В других языках, включая Object Pascal, C++, Ada, CLOS, модуль - это самостоятельная языковая конструкция. В этих языках классы и объекты составляют логическую структуру системы, они помещаются в модули, образующие физическую структуру системы. Это свойство становится особенно полезным, когда система состоит из многих сотен классов.

Согласно Барбаре Лисков "модульность - это разделение программы на фрагменты, которые компилируются по отдельности, но могут устанавливать связи с другими модулями". Мы будем пользоваться определением Парнаса: "Связи между модулями - это их представления друг о друге". В большинстве языков, поддерживающих принцип модульности как самостоятельную концепцию, интерфейс модуля отделен от его реализации. Таким образом, модульность и инкапсуляция ходят рука об руку. В разных языках программирования модульность поддерживается по-разному. Например, в C++ модулями являются отдельно компилируемые файлы. Для C/C++ традиционным является помещение интерфейсной части модулей в отдельные файлы с расширением .h (так называемые файлы-заголовки). Реализация, то есть текст модуля, хранится в файлах с расширением .c (в программах на C++ часто используются расширения .ee, .sr и .crr). Связь между файлами объявляется директивой макропроцессора `#include`. Такой подход строится исключительно на соглашении и не является строгим требованием самого языка. В языке Object Pascal принцип модульности формализован несколько строже. В этом языке определен особый синтаксис для интерфейсной части и реализации модуля (unit). Язык Ada идет еще на шаг дальше: модуль (называемый package) также имеет две части - спецификацию и тело. Но, в отличие от Object Pascal, допускается раздельное определение связей с модулями для спецификации и тела пакета. Таким образом, допускается, чтобы тело модуля имело связи с модулями, невидимыми для его спецификации.

Правильное разделение программы на модули является почти такой же сложной задачей, как выбор правильного набора абстракций. Абсолютно прав Зельковиц, утверждая: "поскольку в начале работы над проектом решения могут быть неясными, декомпозиция на модули может вызвать затруднения. Для хорошо известных приложений (например, создание компиляторов) этот процесс можно стандартизовать, но для новых задач (военные системы или управление космическими аппаратами) задача может быть очень трудной".

Для небольших задач допустимо описание всех классов и объектов в одном модуле. Однако для большинства программ (кроме самых тривиальных) лучшим решением будет сгруппировать в отдельный модуль логически связанные классы и объекты, оставив открытыми те элементы, которые совершенно необ-

ходимо видеть другим модулям. Такой способ разбиения на модули хорош, но его можно довести до абсурда. Рассмотрим, например, задачу, которая выполняется на многопроцессорном оборудовании и требует для координации своей работы механизм передачи сообщений. В больших системах, вполне обычным является наличие нескольких сотен и даже тысяч видов сообщений. Было бы наивным определять каждый класс сообщения в отдельном модуле. При этом не только возникает кошмар с документированием, но даже просто поиск нужных фрагментов описания становится чрезвычайно труден для пользователя. При внесении в проект изменений потребуется модифицировать и перекомпилировать сотни модулей. Этот пример показывает, что скрывание информации имеет и обратную сторону. Деление программы на модули бессистемным образом иногда гораздо хуже, чем отсутствие модульности вообще.

В традиционном структурном проектировании модульность - это искусство раскладывать подпрограммы по кучкам так, чтобы в одну кучку попадали подпрограммы, использующие друг друга или изменяемые вместе. В объектно-ориентированном программировании ситуация несколько иная: необходимо физически разделить классы и объекты, составляющие логическую структуру проекта.

Таким образом, программист должен находить баланс между двумя противоположными тенденциями: стремлением скрыть информацию и необходимостью обеспечения видимости тех или иных абстракций в нескольких модулях. Парнас, Клеменс и Вейс предложили следующее правило: "Особенности системы, подверженные изменениям, следует скрывать в отдельных модулях; в качестве межмодульных можно использовать только те элементы, вероятность изменения которых мала. Все структуры данных должны быть обособлены в модуле; доступ к ним будет возможен для всех процедур этого модуля и закрыт для всех других. Доступ к данным из модуля должен осуществляться только через процедуры данного модуля". Другими словами, следует стремиться построить модули так, чтобы объединить логически связанные абстракции и минимизировать взаимные связи между модулями. Исходя из этого, приведем определение модульности:

**Определение 5. Модульность** - это свойство системы, которая была разложена на внутренне связанные, но слабо связанные между собой модули.

Таким образом, принципы абстрагирования, инкапсуляции и модульности являются взаимодополняющими. Объект логически определяет границы определенной абстракции, а инкапсуляция и модульность делают их физически незыблемыми.

Источник: Объектно-ориентированный анализ и проектирование с примерами приложений на C++ [Гради Буч]

## Пример разделения программы на файлы

Например, определим файл `sum.cpp`, который будет иметь следующий код:

```
int sum(int a, int b)
{
    return a + b;
}
```

Это функция вычисления суммы чисел.

Добавим еще один файл - , который будет содержать объявление функции `sum`:

```
int sum(int, int);
```

И также определим главный файл, который назовем **app.cpp**:

```
#include <iostream>
#include "sum.h"    // подключаем файл sum.h

int main()
{
    int result { sum(5, 4)};
    std::cout << "result = " << result << std::endl;    // result = 9
}
```

Функция `main` вызывает функцию `sum` для вычисления суммы чисел. Но перед использованием функции она должна быть определена или по крайней мере должен быть известен ее заголовок. В прошлых темах объявление функции добавлялось непосредственно в главный и единственный файл программы. Однако если функции определены в отдельных файлах, то более оптимально помещать объявления функций в специальные заголовочные файлы и потом подключать эти файлы. Именно поэтому в начале с помощью директивы `include` подключается файл `sum.h`, который содержит объявление или заголовок функции.

```
#include "sum.h"
```

Файл `sum.h` еще называется заголовочным файлом (header file), так как содержит объявление, заголовок функции. Причем в данном случае предполагается что все файлы располагаются в одном каталоге.

Можно было бы и не подключать файл `sum.h` и вообще не создавать его, а объявление функции поместить непосредственно в файл `app.cpp`. Но при изменении функции может потребоваться изменить и ее объявление. И если функция `sum` используется в нескольких файлах с исходным кодом, то в каждом из этих файлов придется менять ее объявление. В данном же случае достаточно изменить объявление функции в одном файле - `sum.h`.

При компиляции через `g++` необходимо передать все файлы через пробел компилятору:

```
g++ app.cpp sum.cpp -o app
```

То же самое верно и для компиляции через `Clang++`:

```
clang++ app.cpp sum.cpp -o app.exe
```

На выходе будет сгенерирован единый файл `app`.

При работе в Visual Studio заголовочные файлы обычно помещаются в каталог "Headers а при компиляции все файлы автоматически компилируются в один.

Источник: metanit.com

## Иерархия – определение, виды иерархий, примеры

### Иерархия

Прежде чем говорить о наследовании, необходимо определить понятие иерархии с точки зрения ООП.

Так как количество объектов, а соответственно и абстракций, бесконечно, то на интуитивном уровне мы всегда пытаемся как-то обобщить или структурировать эти абстракции. Принято говорить, что:

**Определение 6.** «Иерархия – это упорядочение абстракций, расположение их по уровням. Основными видами иерархических структур применительно к сложным системам являются структура классов (иерархия «is-a») и структура объектов (иерархия «part of»)»

Иерархия «is-a» или «это – разновидность того» служит для обобщения сходных по каким-то характеристикам или поведению классов. Она позволяет выделить то, что является для них общим, и создать на основе этой общности так называемый родительский или базовый класс.

Например, мы автоматически говорим о телевизоре, утюге, стиральной машине или компьютере, что они все являются электроприборами. И, соответственно, ждем, что у всех будет кнопка «вкл/выкл», будет известна потребляемая мощность и допустимое напряжение.

Идея проектирования иерархии наследования в программе состоит в том, что разработчик определяет, какие объекты обладают каким набором общих методов или характеристик, важных для него с точки зрения задачи. Этот набор разработчик описывает как базовый класс, задав для него интерфейс взаимодействия с окружающим миром. Затем для каждого наследника можно создать свой класс, в который будут добавлены специфические для наследника атрибуты и элементы поведения и, возможно, будут изменены в реализации общие методы.

В наследственной иерархии общая часть структуры и поведения сосредоточена в наиболее общем родительском классе. По этой причине говорят о наследовании, как об иерархии «обобщение-специализация». Родительские классы при этом отражают наиболее общие, а наследники – более специализированные абстракции, в которых члены родителя могут быть дополнены, модифицированы и даже скрыты.

Основная проблема при проектировании иерархии – правильно выбрать общие атрибуты и методы, иначе можно получить утюг с клавиатурой или холодильник с функцией глажки.

### Агрегирование

Когда мы определяли понятие иерархии, то говорили о том, что есть иерархия вида "это – часть того". Такая иерархия и определяет агрегирование объектов. Агрегирование используется в тех случаях, когда объект состоит из набора других объектов: самолет собирается из фюзеляжа, шасси, крыльев, двигателей, рулей высоты и т.д.; государство с точки зрения административной структуры может состоять из округов, округа включают в себя области, области – районы и т.д., до уровня населенных пунктов.

Агрегирование может заменить множественное наследование. Например, машину скорой помощи можно описать тогда как машину, имеющую дополнительный функционал и атрибуты, связанные с аварийными службами.

```
class AmbulanceCar : public Car {
    Emergency* em;
public:
    AmbulanceCar(string m);
};
```

В этом случае машина будет работать как обычная машина, во всех случаях, когда она не задействована в аварийных работах.

Агрегирование является мощным средством проектирования программ, так как позволяет собирать объекты из уже готовых частей, имеющих полностью описанный функционал, задача агрегации в этом случае – описать совместную работу всех составных частей.

Агрегировать объекты можно двумя способами: использование (собственно агрегация) и владение (композиция). Если один объект использует другой, то это означает, что объемлющий объект работает со своей частью, но часть может существовать без целого. Например, сотрудник может существовать без организации (он перейдет в другую), если организация будет закрыта. В случае владения часть не может существовать без целого – отдел есть, пока есть организация. Чтобы продемонстрировать разницу между этими способами, рассмотрим следующий пример.

**Пример.** В организации есть несколько отделов, отдел задается своим названием, в отделе есть сотрудники, о них известно имя и должность, сотрудник принимается в конкретный отдел, перед расформированием фирмы или отдела сотрудники должны быть уволены.

```
// класс служащих
class Employer {
    string name;
    string position;
public:
    Employer(string n, string p) {
        name = n;
        position = p;
    };
    // узнать имя сотрудника
    string getName() { return name; };
    // узнать должность сотрудника
    string getPosition() {return position;};
    // задать должность сотрудника
    void setPosition(string p) {position = p;};
};

//класс отделов
class Department {
    string name;
    list<Employer*> lstEmployer; // список сотрудников
```

```

public:
    Department(string n) {
        name = n;
    };
    string getName() { return name; }
    //принять служащего в отдел
    void AddEmployer(Employer* emp);
    // уволить служащего из отдела
    void RemoveEmployer(Employer* emp);
    // Расформировать отдел
    void DelDepartment();
    // Напечатать состав отдела
    void PrintDep();
};

void Department::AddEmployer(Employer* emp) {
    lstEmployer.push_back(emp);
}

void Department::RemoveEmployer(Employer* emp) {
    lstEmployer.remove(emp);
}

// Расформировать отдел
void Department::DelDepartment() {
    // предварительно уволив сотрудников -
    for each (Employer* var in lst Employer)
    {
        // все сотрудники отдела теперь ищут работу
        var->setPosition("ищет работу");
    }
    lstEmployer.clear();
}

void Department::PrintDep() {
    cout << "В отделе " << name << " работают: ";
    for each (Employer* var in lstEmployer)
    {
        cout<< var->getName() << " ";
    }
    cout << '\n';
};

// класс фирм
class Firma {
private:
    string name;
    list<Department*> lstDepartment; //список отделов
public:
    Firma(string n) {

```

```

        name = n;
    };
    // Создать и удалить отдел
    void AddDep(string nDep);
    void RemoveDep(string nDep);
    // Принять и уволить служащих
    void AddEmpl(Employer* emp, string nDep);
    void RemoveEmpl(Employer* emp, string nDep);
    // Расформировать фирму
    void DelFirma();
    void PrintFirma();
};

void Firma::AddDep(string nDep) {
    lstDepartment.push_back(new Department(nDep));
}

void Firma::RemoveDep(string nDep) {
    list<Department*>::iterator it;
    //нашли отдел, который расформировывается
    for ( it = lstDepartment.begin(); it != lstDepartment.end(); it++)
    {
        if ((*it)->getName() == nDep) break;
    };
    // удалили его из фирмы
    it->DelDepartment();
    lstDepartment.remove(*it);
    delete (*it);
};

void Firma::AddEmpl(Employer* emp, string nDep) {
    list<Department*>::iterator it;
    for (it = lstDepartment.begin(); it != lstDepartment.end(); it++)
    {
        if ((*it)->getName() == nDep) break;
    };
    if (it != lstDepartment.end())
        (*it)->AddEmployer(emp);
}

void Firma::RemoveEmpl(Employer* emp, string nDep) {
    list<Department*>::iterator it;
    for (it = lstDepartment.begin(); it != lstDepartment.end(); it++)
    {
        if ((*it)->getName() == nDep) break;
    };
    if (it != lstDepartment.end())
        (*it)->RemoveEmployer(emp);
};

```

```

void Firma::DelFirma() {
    // предварительно расформировав отделы
    for each (Department* var in lstDepartment)
    {
        var->DelDepartment();
    }
    lstDepartment.clear();
};

void Firma::PrintFirma() {
    cout << "Структура фирмы " << name << ": \n";
    for each (Department* var in lstDepartment)
    {
        var->PrintDep();
    }
    cout << '\n';
};

```

В примере видна разница между способами создания и удаления для сотрудников и отделов. На работу принимается в методе AddEmpl уже существующий объект – сотрудник, и когда сотрудник увольняется, то из списка сотрудников просто удаляется ссылка на этот объект, а сам объект остается неизменным. В то же время объект отдел создается внутри фирмы методом AddDep, который задает только имя отдела, и при удалении из списка отделов уничтожается. Когда фирма прекратит свое существование, то отделы будут расформированы, а вот сотрудники (т.е. экземпляры класса Employer)смогут устроиться в другую фирму. И, кстати, сотрудники могут одновременно работать в нескольких организациях на одной и той же должности!

Создадим две фирмы и четырех служащих:

```

Firma f1("Рога и копыта");
Firma f2("Шерсть и перья");
Employer emp1("Иванов", "Директор");
Employer emp2("Петров", "Бухгалтер");
Employer emp3("Сидоров", "Кассир");
Employer emp4("Кузнецов", "Инженер");

```

В фирму «Рога и копыта» добавим отдел «Администрация» и примем туда на работу Иванова и Петрова.

```

f1.AddDep("Администрация");
f1.AddEmpl(&emp1, "Администрация");
f1.AddEmpl(&emp2, "Администрация");
f1.PrintFirma();

```

В фирму «Шерсть и перья» добавим отдел «Бухгалтерия» и примем туда на работу Сидорова и Кузнецова.

```

f2.AddDep("Бухгалтерия");
f2.AddEmpl(&emp3, "Бухгалтерия");
f2.AddEmpl(&emp4, "Бухгалтерия");
f2.PrintFirma();

```

Создадим в фирме «Рога и копыта» отдел «Касса» и примем туда по совместительству кассира Сидорова.

```
f1.AddDep("Касса");  
f1.AddEmpl(&emp3, "Касса");
```

Теперь попробуем удалить фирму "Шерсть и перья" и принять Кузнецова в «Рога и копыта».

```
f2.DelFirma();  
f1.AddEmpl(&emp4, "Администрация");  
f1.PrintFirma();
```

Результат работы представлен ниже:

```
Структура фирмы Рога и копыта:  
В отделе Администрация работают: Иванов Петров  
Структура фирмы Шерсть и перья:  
В отделе Бухгалтерия работают: Сидоров Кузнецов  
Структура фирмы Рога и копыта:  
В отделе Администрация работают: Иванов Петров Кузнецов  
В отделе Касса работают: Сидоров
```

Можно сказать, что при выборе типа иерархии для классов при проектировании приложений разработчик должен понимать, с чем он имеет дело в каждом случае. Если задача предполагает обобщение функционала различных объектов, то выбирается наследование «это – разновидность того», если же предполагается конструирование сложных систем, необходима иерархия «это – часть того». Естественно, что в реальных проектах обе иерархии постоянно используются вместе.

# Наследование, виды наследования, примеры

## Иерархия наследования

Идея проектирования иерархии наследования в программе состоит в том, что разработчик определяет, какие объекты обладают каким набором общих методов или характеристик, важных для него с точки зрения задачи. Этот набор разработчик описывает как базовый класс, задав для него интерфейс взаимодействия с окружающим миром. Затем для каждого наследника можно создать свой класс, в который будут добавлены специфические для наследника атрибуты и элементы поведения и, возможно, будут изменены в реализации общие методы.

В наследственной иерархии общая часть структуры и поведения сосредоточена в наиболее общем родительском классе. По этой причине говорят о наследовании, как об иерархии «обобщение-специализация». Родительские классы при этом отражают наиболее общие, а наследники – более специализированные абстракции, в которых члены родителя могут быть дополнены, модифицированы и даже скрыты.

## Одиночное наследование

**Пример.** Внесем изменения в наш сквозной пример. В зоопарке появился еще один представитель животного мира – павлин. Если бы механизма наследования не было, то пришлось бы описывать все методы, определенные для павлина, заново, а это означает достаточно большое дублирование кода – павлинов также надо кормить, у них есть имена, и они могут двигаться. Правда, павлины еще умеют летать.

Логично будет создать базовый класс `Animalia`, в котором будут собраны общие методы и атрибуты:

```
class Animalia {
private:
    double weight;
    string name;
    void setWeight(double we);
public:
    Animalia (string n, double w); // у разных видов
    // животных различный
    // начальный вес
    void IsEating(double we, string e);
    void Go();
    string getName();
    double getWeight();
};

Animalia::Animalia (string n, double w){
    name = n;
    weight = w;
}
```

```

void Animalia::IsEating(double we, string e){
    if (we > 0) {
        setWeight(we);
        cout << name << " съел " << we << " кг корма " << e
        << "\n";
    }
    else cout << name << " не дали корм\n";
}

void Animalia::setWeight(double we){
    weight = weight + 0.1 * e; // 10% корма на прибавку к весу
}

void Animalia::Go() {
    cout << name << " идет\n";
}

string Animalia::getName() { return name; }
double Animalia::getWeight() { return weight; }

```

Отметим, что создание базового класса немного изменило описание функции кормления животных: пришлось добавить название корма для животного, так как павлины не едят мед, да и медведи на самом деле тоже питаются не только медом. Чем отличаются медведи и павлины в поведении? Павлины умеют летать, а медведям добавим для примера умение ловить рыбу.

Тогда можно определить два класса-потомка: медведи — Bear, павлины — Peacock, и задать для них различающийся функционал.

```

class Bear : public Animalia {
public:
    Bear(string n, double w);
    double TakeFish();
};

class Peacock : public Animalia {
public:
    Peacock(string n, double w);
    void Fly();
};

```

со следующей реализацией:

```

Bear::Bear(string n, double w): Animalia(n,w){
}

double Bear::TakeFish(){
    double weightFish = random()%1000;
    cout << "Медведь " << name << " поймал рыбу весом " << weightFish << " кг\n";
    return weightFish;
}

```

```

Peacock::Peacock (string n, double w): Animalia(n,w){
}

void Peacock:: Fly(){
    cout << "Павлин " << name << " летит " <<"\n";
}

```

При описании наследников после имени класса-потомка ставится :<модификатор доступа> <имя родительского класса>. Эта конструкция определяет тип наследования – **public**, **private** – и от какого базового класса оно выполнено. Если указано **public**-наследование, то все **public** атрибуты и методы базового класса будут иметь в классе-потомке уровень доступа **public**, если указано **private**-наследование, то все **public** методы и атрибуты базового класса попадают в **private** раздел дочернего класса и могут быть доступны только из него.

Поля и атрибуты, отмеченные как **private** в родительском классе, останутся невидимыми для класса-потомка, и доступ к ним будет возможен только через геттеры и сеттеры базового класса.

Особое место занимает модификатор **protected** в описании базового класса: те атрибуты и методы, которые были описаны в разделе **protected**, будут доступны для методов класса-потомка так же, как если бы они находились в разделе **public** базового класса.

Чаще всего **private**-наследование используют в тех случаях, когда надо полностью скрыть устройство базового класса от классов-потомков, создать что-то подобное черному ящику – функционал использовать можно, а доступ к внутренней структуре получить нельзя извне.

Теперь создадим объекты этих классов и посмотрим на работу с ними:

```

Bear bear("Винни", 100);
cout << "Медведь " << bear.getName() << " живет в зоопарке" << '\n';
bear.IsEating(3, "Мед");
bear.Go();
Peacock peacock("Павлуша", 10);
cout << "Павлин " << peacock.getName() << " живет в зоопарке" << '\n';
peacock.IsEating(3, "Трава");
peacock.Go();
peacock.Fly();

```

получим следующий результат:

```

Медведь Винни живет в зоопарке
Винни съел 3 кг корма Мед
Винни идет
Павлин Павлуша живет в зоопарке
Павлуша съел 3 кг корма Трава
Павлуша идет
Павлин Павлуша летит

```

Теперь появляется возможность использовать какой-то общий метод для всех животных зоопарка, например, надо узнать вес и имена всех животных и перевести в другое место. Если бы класса-предка не было, то пришлось бы узнавать вес и имена павлинов и медведей разными фрагментами кода, а в данном случае достаточно написать следующее:

```

Bear bear("Винни", 100);
Peacock peacock("Павлуша", 10);
Bear bear1("Пух", 200);
list<Animalia*> Zoo; //список всех животных зоопарка
Zoo.push_back(&bear); //добавляем в него живущих
Zoo.push_back(&peacock); //сейчас в зоопарке
for each (Animalia* var in Zoo)
{
    var->Go();
    cout << var->getName() << " весит " << var->getWeight() << "кг\n";
}

```

и тогда можно получить полную информацию о животных зоопарка:

```

Винни идет
Винни весит 100.3 кг
Павлуша идет
Павлуша весит 10.3 кг

```

## Множественное наследование

Язык C++ – один из немногих языков программирования, поддерживающих множественное наследование. Множественное наследование подразумевает, что у класса-потомка может быть несколько родителей. Такое возможно, если с точки зрения разработчика проектируемый класс в равной степени обладает поведением двух или более несвязанных друг с другом классов.

**Пример.** Есть класс Car, описывающий машины. Для каждой машины заданы ее марка и скорость движения:

```

class Car {
private:
    string marka;
    int curSpeed;
public:
    Car(string m);
    int getSpeed();
    void setSpeed(int sp);
};

```

и класс Emergency для описания транспорта аварийных служб:

```

class Emergency {
private:
    int status;
    string Color;
protected:
    virtual Go();
public:
    Emergency();
    void Signal();
    void NoAlarm();
    void Alarm();
};

```

Известно, что такой транспорт имеет особую раскраску, звуковой сигнал, особые правила движения по тревоге. Тогда логичным кажется создать машину скорой помощи как потомка этих двух классов, так как они обладают абсолютно разным поведением:

```
class AmbulanceCar : public Car, public Emergency {
public:
    AmbulanceCar(string m);
};
```

В принципе такая реализация удобна. Если надо будет подать сигнал всему аварийному транспорту, то экземпляр класса `AmbulanceCar` также сможет включить сирену, а если необходимо задать скорость движения машин, то это можно сделать обращением к методу, унаследованному от `Car`.

Почему же множественное наследование используется достаточно редко, а многие языки его не поддерживают вовсе?

Во-первых, для правильной работы такого наследования необходима полная независимость классов-родителей друг от друга, а во-вторых, если случайно у родителей совпадут названия методов, то программа не пройдет даже компиляцию.

Например, если для класса `Car` будет определен метод `Go()`, то возникнет конфликт имен – в вызове

```
AmbulanceCar ac("Volvo");
ac.Go();
```

К какому методу обращаемся? Унаследованному от `Car` или от `Emergency`? Необходимо явно указать, к методу какого класса идет обращение:

```
ac.Emergency::Go();
ac.Car::Go();
```

Такая же проблема возможна и с именами атрибутов. Второй проблемой множественного наследования является случай ромбовидного наследования, когда класс-потомок имеет нескольких родителей, а вот у них есть один общий предок. В этом случае все атрибуты и методы общего предка будут включаться в потомка столько раз, сколько раз предок встречается в описании родителей. Единственным способом избежать конфликта является добавление к типу наследования (`public` или `private`) ключевого слова `virtual`:

```
class Base {
    int Id;
public:
    Base();
};

class Derived1 : virtual public Base {
    int Id_d1;
public:
    Derived1();
};
```

```
class Derived2 : virtual public base {
    int Id_d2;
public:
    Derived2();
};

// Derived3 является потомком как Derived1, так и
// Derived2. В Derived3 одна копия класса Base.

class Derived3: public Derived1, public Derived2 {
    int Id_d3;
public:
    Derived3();
};
```

Споры об использовании множественного наследования продолжаются много лет. С одной стороны, это повышает гибкость кода, с другой – может способствовать возникновению ошибок даже в ранее работавшем коде, если вдруг у одного из базовых классов появятся методы или атрибуты с именами, совпадающими с именами из другого базового класса. Это вполне возможно, так как базовые классы "не подозревают" о существовании друг друга. Поэтому многие проектировщики придерживаются мнения, что множественное наследование может быть заменено на сочетание одиночного наследования и агрегирования.

## Реализация полиморфизма в языках программирования. Виртуальные методы

Если бы все преимущества наследования ограничивались бы только возможностью повторного использования кода и возможностью определения одинакового поведения для всех родственных объектов, то ООП не приобрело бы такой популярности среди разработчиков.

Наследование позволяет, с одной стороны, обращаться с объектами наследниками, как с объектами базового класса, с другой стороны, реализовывать эти действия по-своему.

В классах-потомках возможно переопределение общего функционала класса, если это необходимо.

**Пример.** Изменим реализацию метода `Go()` для павлина. Павлин будет двигаться вприпрыжку, а не шагом.

Тогда в описание класса `Peacock` добавится метод `Go` со следующей реализацией:

```
void Peacock::Go() {
    cout << "Павлин " << " " << getName() << "подпрыгивает на ходу\n";
}
```

и тот же код, что и в предыдущих двух примерах даст другой результат:

```
Медведь Винни живет в зоопарке
Винни съело 3 кг корма мед
Винни идет
Павлин Павлуша живет в зоопарке
Павлуша съел 3 кг корма Трава
Павлуша подпрыгивает на ходу
Павлин Павлуша летит
Винни идет
Винни весит 100.3 кг
Павлуша идет
Павлуша весит 10.3 кг
```

Видно, что когда обращение было напрямую к объекту класса `Peacock`, то двигаться павлин стал по-другому, а вот когда обратились к нему, как к разновидности животных, то движение стало прежним.

Произошло так называемое скрытие метода дочернего класса при обращении через родительский класс. Имена совпадают, а тип объекта определяется типом объекта, через который происходит обращение, в данном случае это тип `Animalia`, поэтому были «потеряны» все индивидуальные особенности класса `Peacock`, они заменились общими методами.

Чтобы исправить эту ситуацию, необходимо сообщить компилятору, что надо проверять реальный тип объекта при обращении к его методам. Для этого те методы, которые хотим вызывать для сохранения специфики объекта, надо объявлять с ключевым словом `virtual` в родительском классе:

```

class Animalia {
...
public:
...
virtual void Go();
...
};

```

В этом случае, при компиляции будет определен реальный тип объекта и будет выполнена его реализация метода:

```

Медведь Винни живет в зоопарке
Винни съел 3 кг корма мед
Винни идет
Павлин Павлуша живет в зоопарке
Павлуша съел 3 кг корма Трава
Павлин Павлуша подпрыгивает на ходу
Павлин Павлуша летит
Винни идет
Винни весит 100.3 кг
Павлуша подпрыгивает на ходу
Павлуша весит 10.3 кг

```

Обычно ключевое слово `virtual` добавляют сразу и для методов дочернего класса, на случай, если придется создавать более разветвленную иерархию наследования, чтобы в дальнейшем не забыть об этом.

**Определение 7.** Методы, описанные со словом `virtual`, называются **виртуальными**.

**Определение 8.** Возможность использовать для объектов с одинаковым описанием различные реализации называется **полиморфизмом**.

Именно полиморфизм позволяет создавать гибкий код, который будет работать с каждым объектом в программе так, как необходимо этому объекту.

Если по каким-то причинам надо обратиться из потомка к методу именно родительского класса, то перед вызовом метода указывается имя класса:

```

peacock.Animalia::Go();

```

## Абстрактные и виртуальные классы

**Определение 9.** Методы, описанные со словом `virtual`, называются **виртуальными**.

При проектировании иерархии наследования очень часто разработчики сталкиваются с ситуацией, когда для родительского класса можно сформулировать интерфейс, указав все необходимые методы, но нельзя сказать, как они будут реализовываться. Например, если создать иерархию геометрических фигур, то понятно, что должен быть метод вычисления площади фигуры, но определить, как это делается для произвольной фигуры, невозможно.

Одним из вариантов решения проблемы является создание пустых методов или методов, возвращающих ноль:

```
void Figure::Print() {}
double Figure::Area(){ return 0; }
```

Но это неправильный путь, который может привести к большим ошибкам в программе, если кто-то создаст объект родительского класса, рассчитает площадь, а потом разделит (или, еще хуже, умножит) на неё другое число. Поэтому в объектно-ориентированных языках программирования введены понятия **чисто виртуального метода** и абстрактного класса.

**Определение 10.** Метод является чисто виртуальным, если имя метода приравнивается к нулю:

```
class Animalia {
public:
    ...
    virtual void IsEating(string e, double we) = 0;
    virtual void Go() = 0;
    ...
};
```

В этом случае нельзя будет задать реализацию метода в классе и, значит, нельзя будет создать экземпляр этого класса. Так делают в тех случаях, когда родительский класс является настолько общим, что создание экземпляра этого класса не имеет смысла.

**Определение 11.** Классы, содержащие чисто виртуальные методы, называются **абстрактными классами**.

Все классы-наследники абстрактного класса должны переопределить у себя чисто виртуальный метод, если метод останется не переопределенным, то соответствующий класс также будет абстрактным.

## Виртуальные классы?

Аккуратно: источник — Википедия

**Определение 12. Виртуальное наследование** (англ. virtual inheritance) в языке программирования C++ — один из вариантов наследования, который нужен для решения некоторых проблем, порождаемых наличием возможности множественного наследования (особенно «ромбовидного наследования»), путём разрешения неоднозначности того, методы которого из суперклассов (непосредственных классов-предков) необходимо использовать. Оно применяется в тех случаях, когда множественное наследование вместо предполагаемой полной композиции свойств классов-предков приводит к ограничению доступных наследуемых свойств вследствие неоднозначности. Базовый класс, наследуемый множественно, определяется виртуальным с помощью ключевого слова **virtual**.

### Пример

```
#include <iostream>

class A {
public:
    virtual int foo() {
        return 1;
    }
};

class B : public virtual A {};

class C : public virtual A {};

class D : public B, public C {};

int main () {
    D d;
    std::cout << d.foo();
    return 0;
}
```

Если убрать ключевое слово `virtual`, то метод `foo()` не может быть определён однозначно и в результате не будет доступен, как и объект класса `D` — код не скомпилируется.

## Отличия виртуальной функции от чисто виртуальной

| ВИРТУАЛЬНАЯ<br>ФУНКЦИЯ                                                                                               | ЧИСТАЯ ВИРТУАЛЬНАЯ<br>ФУНКЦИЯ                                                                                      |
|----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Функция-член, объявленная в базовом классе, которая может быть переопределена или переопределена производным классом | Виртуальная функция в базовом классе без реализации                                                                |
| Базовый класс имеет определение функции                                                                              | Базовый класс не имеет определения функции - он имеет только объявление функции                                    |
| Нет необходимости для всех производных классов переопределять виртуальную функцию базового класса                    | Производные классы должны переопределять чисто виртуальную функцию базового класса                                 |
| Нет абстрактной концепции класса                                                                                     | Если существует хотя бы одна чисто виртуальная функция, этот класс называется абстрактным классом                  |
| Производный класс, который не переопределяет виртуальную функцию базового класса, не вызывает ошибок компиляции      | Производный класс, который не переопределяет чисто виртуальную функцию базового класса, вызывает ошибки компиляции |
|                                                                                                                      | <a href="http://www.rznisa.ru">www.rznisa.ru</a>                                                                   |

## Типизация. Статическое и динамическое связывание

Что такое типизация? Понятие типа взято из теории абстрактных типов данных. Дойч определяет тип, как "точную характеристику свойств, включая структуру и поведение, относящуюся к некоторой совокупности объектов". Для наших целей достаточно считать, что термины тип и класс взаимозаменяемы [Тип и класс не вполне одно и то же; в некоторых языках их различают. Например, ранние версии языка Trellis/Owl разрешали объекту иметь и класс, и тип. Даже в Smalltalk объекты классов SmallInteger, LargeNegativeInteger, LargePositiveInteger относятся к одному типу Integer, хотя и к разным классам. Большинству смертных различать типы и классы просто противно и бесполезно. Достаточно сказать, что класс реализует понятие типа]. Тем не менее, типы стоит обсудить отдельно, поскольку они выставляют смысл абстрагирования в совершенно другом свете. В частности, мы утверждаем, что:

**Определение 13. Типизация** - это способ защититься от использования объектов одного класса вместо другого, или по крайней мере управлять таким использованием.

Типизация заставляет нас выражать наши абстракции так, чтобы язык программирования, используемый в реализации, поддерживал соблюдение принятых проектных решений. Вегнер замечает, что такой способ контроля существенен для программирования "в большом".

Идея согласования типов занимает в понятии типизации центральное место. Например, возьмем физические единицы измерения. Деля расстояние на время, мы ожидаем получить скорость, а не вес. В умножении температуры на силу смысла нет, а в умножении расстояния на силу - есть. Все это примеры сильной типизации, когда прикладная область накладывает правила и ограничения на использование и сочетание абстракций.

**Примеры сильной и слабой типизации.** Конкретный язык программирования может иметь сильный или слабый механизм типизации, и даже не иметь вообще никакого, оставаясь объектно-ориентированным. Например, в Eiffel соблюдение правил использования типов контролируется непреклонно, - операция не может быть применена к объекту, если она не зарегистрирована в его классе или суперклассе. В сильно типизированных языках нарушение согласования типов может быть обнаружено во время трансляции программы. С другой стороны, в Smalltalk типов нет: во время исполнения любое сообщение можно послать любому объекту, и если класс объекта (или его надкласс) не понимает сообщение, то генерируется сообщение об ошибке. Нарушение согласования типов может не обнаружиться во время трансляции и обычно проявляется как ошибка исполнения. C++ тяготеет к сильной типизации, но в этом языке правила типизации можно игнорировать или подавить полностью.

**Примеры типизации: статическое и динамическое связывание.**

Сильная и статическая типизация - разные вещи. Строгая типизация следит за соответствием типов, а статическая типизация (иначе называемая статическим или ранним связыванием) определяет время, когда имена связываются с типами.

Статическая связь означает, что типы всех переменных и выражений известны во время компиляции; динамическое связывание (называемое также позд-

ним связыванием) означает, что типы неизвестны до момента выполнения программы.

Концепции типизации и связывания являются независимыми, поэтому в языке программирования может быть: типизация - сильная, связывание - статическое (Ada), типизация - сильная, связывание - динамическое (C++, Object Pascal), или и типов нет, и связывание динамическое (Smalltalk). Язык CLOS занимает промежуточное положение между C++ и Smalltalk: определения типов, сделанные программистом, могут быть либо приняты во внимание, либо не приняты.

Полиморфизм возникает там, где взаимодействуют наследование и динамическое связывание. Это одно из самых привлекательных свойств объектно-ориентированных языков (после поддержки абстракции), отличающее их от традиционных языков с абстрактными типами данных. Противоположность полиморфизму называется мономорфизмом; он характерен для языков с сильной типизацией и статическим связыванием (Ada).

Источник: Объектно-ориентированный анализ и проектирование с примерами приложений на C++ [Гради Буч]

## Пример статической типизации

C++ является статически типизированным языком программирования. То есть если мы определили для переменной какой-то тип данных, то в последующем мы этот тип изменить не сможем. Соответственно переменная может получить значения только того типа, который она представляет. Однако нередко возникает необходимость присвоить переменной значения каких-то других типов. И в этом случае применяются преобразования типов.

Ряд преобразований компилятор может производить неявно, то есть автоматически. Например:

```
#include <iostream>
int main()
{
    unsigned int age{25};
    std::cout << "age = " << age << std::endl;
}
```

Здесь переменная `age` представляет тип `unsigned int` и условно хранит возраст. Эта переменная инициализируется числом 25, а все целочисленные литералы без суффиксов по умолчанию представляют тип `int` (`signed int`). Но компилятор знает как преобразовать значение 25 к типу `unsigned int`, и каких-то проблем в данном случае не будет.

Но посмотрим на другой пример:

```
#include <iostream>
int main()
{
    unsigned int age{-25};
    std::cout << "age = " << age << std::endl;
}
```

Здесь переменной `age` уже присваивается число -25 - отрицательное, в то время как тип переменной - `unsigned int` предполагает лишь использование положительных чисел. И в этом случае мы столкнемся с ошибкой компиляции.

## Пример динамического связывания при передаче параметров

Если необходимо обеспечить динамическое связывание при передаче параметров в функцию, то такой параметр должен представлять ссылку или указатель на объект базового типа:

```
#include <iostream>

class Person
{
public:
    Person(std::string name): name{name} { }
    virtual void print() const // виртуальная функция
    {
        std::cout << name << std::endl;
    }
    std::string getName() const {return name;}
private:
    std::string name;
};

class Employee: public Person
{
public:
    Employee(std::string name, std::string company):
        Person{name}, company{company}{ }
    void print() const override // функция переопределена
    {
        std::cout << getName() << " (" << company << ")" << std::endl;
    }
private:
    std::string company;
};

void printPerson(const Person& person)
{
    person.print();
}

int main()
{
    Person tom {"Tom"};
    Employee bob {"Bob", "Microsoft"};
    printPerson(tom); // Tom
    printPerson(bob); // Bob (Microsoft)
}
```

В данном случае функция `printPerson` в качестве параметра принимает константную ссылку на объект типа `Person`, коим в реальности также может быть объект `Employee`. Поэтому при вызове функции `print` программа будет динамически решать, какую именно реализацию функции вызывать.

## Пример динамического связывания в коллекции

Объекты базовых и производных классов можно хранить в одной коллекции, например, массиве. Например:

```
#include <iostream>

class Person
{
public:
    Person(std::string name): name{name} { }
    virtual void print() const // виртуальная функция
    {
        std::cout << name << std::endl;
    }
    std::string getName() const {return name;}
private:
    std::string name;
};

class Employee: public Person
{
public:
    Employee(std::string name, std::string company):
        Person{name}, company{company}{ }
    void print() const override // функция переопределена
    {
        std::cout << getName() << " (" << company << ")" << std::endl;
    }
private:
    std::string company;
};

void printPerson(const Person& person)
{
    person.print();
}

int main()
{
    Person tom {"Tom"};
    Employee bob {"Bob", "Microsoft"};
    Employee sam {"Sam", "Google"};
    Person people[]{tom, bob, sam};
    for(const auto& person: people)
    {
        person.print();
    }
}
```

Здесь массив `people` хранит объекты `Person`, в качестве которых также могут выступать объекты `Employee`. Однако при такой организации каждый объект

Employee, который помещается в массив, преобразуется в объект Person. В итоге при переборе такого массива вызывается функция `print` из класса Person:

```
Tom
Bob
Sam
```

Если мы хотим обеспечить для элементов массива динамическое связывание, то такие объекты должны представлять указатели. Например, используем указатели:

```
int main()
{
    Person tom {"Tom"};
    Employee bob {"Bob", "Microsoft"};
    Employee sam {"Sam", "Google"};
    Person* people[] { &tom, &bob, &sam }; // массив указателей
    for(const auto& person: people)
    {
        person->print();
    }
}
```

Здесь массив хранит адреса всех объектов, соответственно получим совсем другой вывод:

```
Tom
Bob (Microsoft)
Sam (Google)
```

## Статические члены класса. Дружественные члены класса

### Дружественные функции

В некоторых случаях разработчику необходимо предоставить доступ к полям класса функциям, которые к классу никакого отношения не имеют (один из самых часто встречающихся случаев – перегрузка операций ввода-вывода). Для этого в языке C++ предусмотрены дружественные функции. Они описываются в разделе `public` класса со спецификатором `friend`.

```
class Employer {
    ...
public:
    ...
    friend ifstream& operator >>(ifstream& is, Employer&
    emp);
    friend ofstream& operator <<(ofstream& os, Employer
    emp);
}

ifstream& operator >>(ifstream& is, Employer& emp){
    is >> emp.name >> emp.position;
    return is;
}

ofstream& operator <<(ofstream& os, Employer emp){
    os << emp.name << " " << emp.position;
    return os;
}

// Пример обращения к дружественным функциям.
ifstream in("data.txt");
ofstream out("res.txt");
Employer emp1;
in >> emp1;
out << emp1;
```

Насколько оправдано использование дружественных функций? Многие языки программирования в принципе не поддерживают этот механизм, так как помимо удобства он создает и определенный риск. Дружественная функция получает полный доступ к классу, следовательно, если изменится её реализация, то данные могут быть испорчены.

### Статические члены класса

Когда создается экземпляр класса, то каждый экземпляр имеет свой набор значений характеристик и методов. Однако, практически все объектноориентированные языки программирования предусматривают еще один тип членов класса – статические поля и методы класса.

В коде они описываются со словом `static` перед указанием типа поля или типа возвращаемого значения метода класса. Их смысл в том, что статический член класса один для всех создающихся в программе экземпляров класса.

Например, в класс `Animalia` можно добавить статическое поле `countAnimalia`, в котором будет храниться количество созданных экземпляров класса. При создании очередного животного количество будет увеличиваться, а при вызове деструктора – уменьшаться.

Тогда описание класса `Animalia` примет вид:

```
class Animalia {
    static int countAnimalia;
    string name;
    double weight;
public:
    Animalia () {
        countAnimalia++;
    }

    Animalia (string n, double w) {
        name = n;
        weight = w;
        countAnimalia++;
    }

    ....

    ~Animalia () {
        countAnimalia--;
    }
}
```

Инициализация статической переменной происходит вне класса:

```
int Animalia::countAnimalia = 0;
```

Одним из примеров частого использования статических переменных является случай ограничения доступа к какому-нибудь ресурсу. Например, все служащие имеют право печатать на принтере, но он один и задания можно отправлять на печать только по очереди. Тогда имеет смысл завести статическую переменную `printerLock`, которая будет отражать статус занятости принтера и будет видна всем служащим.

```
class Employer {
    static bool printerLock;
    ...
public:
    ...
    void printDoc (string nameDoc);
}

void Employer :: printDoc(string nameDoc) {
```

```

    if (!printerLock) {
        printerLock = true;
        // печать документа
    }
    else cout << "Принтер занят!\n";
}

```

Если статическое поле описано в разделе `public` класса, то к нему можно обратиться из текста основной программы, если же такое поле описано в разделе `private` (что более логично), то возникает вопрос, как обратиться к такому полю? В принципе язык поддерживает возможность обращения через обычные методы класса, но это не очень верно с точки зрения концепции инкапсуляции и защиты данных, поэтому для работы с такими полями обычно используют методы класса, которые также описываются со словом `static`.

```

class Animalia {
    static int countAnimalia;
    ...
public:
    static getCount(){return countAniamlia;}
    ....
}
int main(){
    Bear bear("Винни",100);
    Peacock peacock("Павлуша", 10);
    Bear bear1("Пух", 200);
    cout << "Всего животных " << Animalia::getCount() << '\n';
}

```

Необходимо помнить, что из статических функций нельзя обращаться к нестатическим полям класса.

## Перегрузка операторов

Перегрузка операторов (operator overloading) позволяет определить для объектов классов втроенные операторы, такие как +, -, \* и т.д. Для определения оператора для объектов своего класса, необходимо определить функцию, название которой содержит слово operator и символ перегружаемого оператора. Функция оператора может быть определена как член класса, либо вне класса.

Перегрузить можно только те операторы, которые уже определены в C++. Создать новые операторы нельзя. Также нельзя изменить количество операндов, их ассоциативность, приоритет.

Если функция оператора определена как отдельная функция и не является членом класса, то количество параметров такой функции совпадает с количеством операндов оператора. Например, у функции, которая представляет унарный оператор, будет один параметр, а у функции, которая представляет бинарный оператор, - два параметра. Если оператор принимает два операнда, то первый операнд передается первому параметру функции, а второй операнд - второму параметру. При этом как минимум один из параметров должен представлять тип класса.

Формальное определение операторов в виде функций-членов класса:

```
// бинарный оператор
ReturnType operator Op(Type right_operand);
// унарный оператор
ClassType& operator Op();
```

Формальное определение операторов в виде функций, которые не являются членами класса:

```
// бинарный оператор
ReturnType operator Op(const ClassType& left_operand, Type right_operand);
// альтернативное определение, где класс, для которого создается оператор,
// представляет правый операнд
ReturnType operator Op(Type left_operand, const ClassType& right_operand);
// унарный оператор
ClassType& operator Op(ClassType& obj);
```

Здесь ClassType представляет тип, для которого определяется оператор. Type - тип другого операнда, который может совпадать, а может и не совпадать с первым. ReturnType - тип возвращаемого результата, который также может совпадать с одним из типов операндов, а может и отличаться. Op - сама операция.

Рассмотрим пример с классом Counter, который хранит некоторое число:

```
#include <iostream>

class Counter
{
public:
    Counter(int val)
    {
        value =val;
```

```

    }
    void print()
    {
        std::cout << "Value: " << value << std::endl;
    }
    Counter operator + (const Counter& counter) const
    {
        return Counter{value + counter.value};
    }
private:
    int value;
};

int main()
{
    Counter c1{20};
    Counter c2{10};
    Counter c3 = c1 + c2;
    c3.print();    // Value: 30
}

```

Здесь в классе Counter определен оператор сложения, цель которого сложить два объекта Counter:

```

Counter operator + (const Counter& counter) const
{
    return Counter{value + counter.value};
}

```

Текущий объект будет представлять левый операнд операции. Объект, который передается в функцию через параметр counter, будет представлять правый операнд операции. Здесь параметр функции определен как константная ссылка, но это необязательно. Также функция оператора определена как константная, но это тоже не обязательно.

Результатом оператора сложения является новый объект Counter, в котором значение value равно сумме значений value обоих операндов.

После определения оператора можно складывать два объекта Counter:

```

Counter c1{20};
Counter c2{10};
Counter c3 {c1 + c2};
c3.print();    // Value: 30

```

Подобным образом можно определить функцию оператора вне класса:

```

#include <iostream>

class Counter
{
public:
    Counter(int val)

```

```

    {
        value =val;
    }
    void print()
    {
        std::cout << "Value: " << value << std::endl;
    }
    int value;
    // к приватным переменным внешняя функция оператора не может обращаться
};
// определяем оператор сложения вне класса
Counter operator + (const Counter& c1, const Counter& c2)
{
    return Counter{c1.value + c2.value};
}

int main()
{
    Counter c1{20};
    Counter c2{10};
    Counter c3 {c1 + c2};
    c3.print();    // Value: 30
}

```

Если бинарный оператор определяется в виде внешней функции, как здесь, то он принимает два параметра. Первый параметр будет представлять левый операнд операции, а второй параметр - правый операнд.

Но по сравнению с предыдущим кодом здесь сделано еще пару изменений. Во-первых, внешняя функция естественно не может обращаться к приватным полям класса, поэтому для доступа к ним придется создавать отдельные функции, которые бы возвращали значения полей. Я для простоты просто сделал переменную `value` публичной. Другим решением в данном случае могло быть определение дружественной функции оператора. Второй момент - внешние функции оператора не могут быть константными. Поэтому гораздо определение операторов внутри класса имеет некоторые преимущества.

Стоит отметить, что необязательно возвращать объект класса. Это может быть и любой объект в зависимости от ситуации. И также мы можем определять дополнительные перегруженные функции операторов:

```

#include <iostream>

class Counter
{
public:
    Counter(int val)
    {
        value =val;
    }
    void print()
    {
        std::cout << "Value: " << value << std::endl;
    }
}

```

```

    }
    Counter operator + (const Counter& counter) const
    {
        return Counter{value + counter.value};
    }
    int operator + (int number) const
    {
        return value + number;
    }
private:
    int value;
};

int main()
{
    Counter counter{20};
    int number = counter + 30;
    std::cout << number << std::endl;    // 50
}

```

Здесь определена вторая версия оператора сложения, которая складывает объект Counter с числом и возвращает также число. Поэтому левый операнд операции должен представлять тип Counter, а правый операнд - тип int.

Какие операторы где переопределять? Операторы присвоения, индексирования ([]), вызова (()), доступа к члену класса по указателю (->) следует определять в виде функций-членов класса. Операторы, которые изменяют состояние объекта или непосредственно связаны с объектом (инкремент, декремент), обычно также определяются в виде функций-членов класса. Операторы выделения и удаления памяти (new new[] delete delete[]) определяются только в виде функций, которые не являются членами класса. Все остальные операторы можно определять как отдельные функции, а не члены класса.

## Операторы сравнения

Результатом операторов сравнения (==, !=, <, >), как правило, является значение типа bool. Например, перегрузим данные операторы для типа Counter:

```

#include <iostream>

class Counter
{
public:
    Counter(int val)
    {
        value =val;
    }
    void print()
    {
        std::cout << "Value: " << value << std::endl;
    }
}

```

```

    bool operator == (const Counter& counter) const
    {
        return value == counter.value;
    }
    bool operator != (const Counter& counter) const
    {
        return value != counter.value;
    }
    bool operator > (const Counter& counter) const
    {
        return value > counter.value;
    }
    bool operator < (const Counter& counter) const
    {
        return value < counter.value;
    }
private:
    int value;
};

int main()
{
    Counter c1(20);
    Counter c2(10);
    bool b1 = c1 == c2;    // false
    bool b2 = c1 > c2;    // true

    std::cout << "c1 == c2 = " << std::boolalpha << b1 << std::endl;
    // c1 == c2 = false
    std::cout << "c1 > c2 = " << std::boolalpha << b2 << std::endl;
    // c1 > c2 = true
}

```

Если речь идет о простом сравнении на основе полей класса, то для операторов == и != проще использовать специальный оператор default:

```

#include <iostream>

class Counter
{
public:
    Counter(int val)
    {
        value =val;
    }
    void print()
    {
        std::cout << "Value: " << value << std::endl;
    }
    bool operator == (const Counter& counter) const = default;

```

```

    bool operator != (const Counter& counter) const = default;
private:
    int value;
};

int main()
{
    Counter c1(20);
    Counter c2(10);
    bool b1 = c1 == c2;    // false
    bool b2 = c1 != c2;    // true

    std::cout << "c1 == c2 = " << std::boolalpha << b1 << std::endl;
    // c1 == c2 = false
    std::cout << "c1 != c2 = " << std::boolalpha << b2 << std::endl;
    // c1 != c2 = true
}

```

Например, в случае с оператором ==:

```

bool operator == (const Counter& counter) const = default;

```

По умолчанию будут сравниваться все поля класса, для которых определен оператор ==. Если значения всех полей будут равны, то оператор возвратит true

## Операторы присвоения

Оператор присвоения обычно возвращает ссылку на свой левый операнд:

```

#include <iostream>

class Counter
{
public:
    Counter(int val)
    {
        value =val;
    }
    void print()
    {
        std::cout << "Value: " << value << std::endl;
    }
    // оператор присвоения
    Counter& operator += (const Counter& counter)
    {
        value += counter.value;
        return *this;    // возвращаем ссылку на текущий объект
    }
private:

```

```

    int value;
};

int main()
{
    Counter c1{20};
    Counter c2{50};
    c1 += c2;
    c1.print();    // Value: 70
}

```

## Унарные операции

Унарные операции обычно возвращают новый объект, созданный на основе имеющегося. Например, возьмем операцию унарного минуса:

```

#include <iostream>

class Counter
{
public:
    Counter(int val)
    {
        value =val;
    }
    void print()
    {
        std::cout << "Value: " << value << std::endl;
    }
    // оператор унарного минуса
    Counter operator - () const
    {
        return Counter{-value};
    }
private:
    int value;
};

int main()
{
    Counter c1{20};
    Counter c2 = -c1;    // применяем оператор унарного минуса
    c2.print();    // Value: -20
}

```

Здесь операция унарного минуса возвращает новый объект Counter, значение value в котором фактически равно значению value текущего объекта, умноженного на -1.

## Операции инкремента и декремента

Особую сложность может представлять переопределение операций инкремента и декремента, поскольку нам надо определить и префиксную, и постфиксную форму для этих операторов. Определим подобные операторы для типа Counter:

```
#include <iostream>

class Counter
{
public:
    Counter(int val)
    {
        value =val;
    }
    void print()
    {
        std::cout << "Value: " << value << std::endl;
    }
    // префиксные операторы
    Counter& operator++ ()
    {
        value += 1;
        return *this;
    }
    Counter& operator-- ()
    {
        value -= 1;
        return *this;
    }
    // постфиксные операторы
    Counter operator++ (int)
    {
        Counter copy {*this};
        ++(*this);
        return copy;
    }
    Counter operator-- (int)
    {
        Counter copy {*this};
        --(*this);
        return copy;
    }
private:
    int value;
};

int main()
{
    Counter c1{20};
```

```

    Counter c2 = c1++;
    c2.print();      // Value: 20
    c1.print();      // Value: 21
    --c1;
    c1.print();      // Value: 20
}

```

Префиксные операторы должны возвращать ссылку на текущий объект, который можно получить с помощью указателя `this`:

```

Counter& operator++ ()
{
    value += 1;
    return *this;
}

```

В самой функции можно определить некоторую логику по инкременту значения. В данном случае значение `value` увеличивается на 1.

Постфиксные операторы должны возвращать значение объекта до инкремента, то есть предыдущее состояние объекта. Поэтому постфиксная форма возвращает копию объекта до инкремента:

```

Counter operator++ (int)
{
    Counter copy {*this};
    ++(*this);
    return copy;
}

```

Чтобы постфиксная форма отличалась от префиксной постфиксные версии получают дополнительный параметр типа `int`, который не используется. Хотя в принципе мы можем его использовать.

## Переопределение оператора «

Оператор « принимает два аргумента: ссылку на объект потока (левый операнд) и фактическое значение для вывода (правый операнд). Затем он возвращает новую ссылку на поток, которую можно передать при следующем вызове оператора « в цепочке.

```

#include <iostream>

class Counter
{
public:
    Counter(int val)
    {
        value =val;
    }
    int getValue()const {return value;}
private:

```

```

    int value;
};

std::ostream& operator<<(std::ostream& stream, const Counter& counter)
{
    stream << "Value: ";
    stream << counter.getValue();
    return stream;
}

int main()
{
    Counter counter1{20};
    Counter counter2{50};
    std::cout << counter1 << std::endl;    // Value: 20
    std::cout << counter2 << std::endl;    // Value: 50
}

```

Стандартный выходной поток `cout` имеет тип `std::ostream`. Поэтому первый параметр (левый операнд) представляет объект `ostream`, а второй (правый операнд) - выводимый объект `Counter`. Поскольку мы не можем изменить стандартное определение `std::ostream`, поэтому определяем функцию оператора, которая не является членом класса.

```

std::ostream& operator<<(std::ostream& stream, const Counter& counter)
{
    stream << "Value: ";
    stream << counter.getValue();
    return stream;
}

```

В данном случае для выводим значение переменной `value`. Для получения значения `value` извне класса `Counter` я добавил функцию `getValue()`.

Возвращаемое значение всегда должно быть ссылкой на тот же объект потока, на который ссылается левый операнд оператора.

После определения функции оператора можно выводить на консоль объекты `Counter`:

```

Counter counter1{20};
std::cout << counter1 << std::endl;    // Value: 20

```

## Выражение одних операторов через другие

Иногда более оптимально выражать одни операторы через другие, нежели создавать отдельно операторы с повторяющейся логикой. Например:

```

#include <iostream>

class Counter
{
public:

```

```

Counter(int n)
{
    value = n;
}
void print() const
{
    std::cout << "value: " << value << std::endl;
}
Counter& operator+=(const Counter& counter)
{
    value += counter.value;
    return *this;
};
Counter& operator+(const Counter& counter)
{
    Counter copy{ value };    // копируем данные текущего объекта
    return copy += counter;
};
private:
    int value;
};

int main()
{
    Counter counter1{20};
    Counter counter2{10};

    counter1 += counter2;
    counter1.print();    // value: 30
    Counter counter3 {counter1 + counter2};
    counter3.print();    // value: 40
}

```

Здесь вначале реализован оператор сложения с присвоением +=:

```

Counter& operator+=(const Counter& counter)
{
    value += counter.value;
    return *this;
};

```

В функции оператора сложения мы создаем копию текущего объекта и к этой копии и аргументу применяем оператор +=:

```

Counter& operator+(const Counter& counter)
{
    Counter copy{ value };    // копируем данные текущего объекта
    return copy += counter;
};

```

В данном случае суть сложения: к полю value прибавляем значение value другого объекта. Однако логика оператора может быть более сложной, и чтобы не повторяться, мы можем таким образом выражать одни операторы через другие.

## Реализация приведения типов в C++

Преобразование или приведение типов для объектно-ориентированного программирования является одним из инструментов реализации наследования и полиморфизма. С одной стороны, это динамическое преобразование типов (или динамическая типизация) при обращении к объектам одного класса как к объектам другого родственного класса:

```
Peacock peacock("Павлуша", 10);
Animalia* a1 = & peacock;
a1->Go();
cout << a1->getName() << " весит " << a1->getWeight() << " кг\n";
Павлуша подпрыгивает на ходу
Павлуша весит 10.3 кг
```

С другой стороны, в некоторых случаях может понадобиться явное преобразование объекта родительского класса к объекту дочернего класса. Естественно, это не всегда возможно, поэтому в C++ существует преобразование `dynamic_cast`, которое используется именно для безопасного преобразования по иерархии наследования. Рассмотрим пример с зоопарком:

```
Bear bear("Винни", 100);
Peacock peacock("Павлуша", 10);
Bear bear1("Пух", 200);
list<Animalia*> Zoo; //список всех животных зоопарка
Zoo.push_back(&bear); //добавляем в него живущих
Zoo.push_back(&peacock); //сейчас в зоопарке
```

Цикл, в котором животные зоопарка двигаются, не требует преобразования типов, так как функция движения существует для всех.

```
for each (Animalia* var in Zoo) var->Go();
```

Если же потребовать, чтобы выполнялись функции, специфические для каждого класса (медведь ловит рыбу, а павлин летит), то методы `TakeFish` и `Fly` будут недоступны для переменной `var`. И даже если к ним обратиться, то во время компиляции возникнет ошибка, так как при полиморфизме вызывается переопределенная общая функция, а не специфическая функция класса. Поэтому воспользуемся преобразованием типов следующим образом:

```
for each (Animalia* var in Zoo)
{
    var->Go();
    Bear *b = dynamic_cast<Bear*>(var);
    if (b != NULL)
        b->TakeFish();
    else {
        Peacock *p = dynamic_cast<Peacock*>(var);
        if (p != NULL)
            p->Fly();
    }
}
```

Если удастся осуществить необходимое преобразование, то значением соответствующего указателя (**b** или **p**) будет адрес в памяти объекта соответствующего класса, и, следовательно, можно будет обратиться к нужному методу. Если преобразование будет безуспешным, то указатель получит значение **NULL**.

Существует еще один интересный способ узнать, с объектом какого класса сейчас идет работа в программе. Если подключить к файлу программы заголовочный файл `<typeid.h>`, то можно будет воспользоваться оператором `typeid`, который возвращает фактический тип объекта (по сути имя класса):

```
string clName;
for each (Animalia* var in Zoo)
{
    clName = typeid(var).name(); // узнали имя класса
    cout<< clName <<"\n"; // напечатали
    var->Go();
    Bear *b = dynamic_cast<Bear*>(var);
    if (clName == "class Bear")
        ((Bear)var)->TakeFish();
    else {
        if (clName == "class Peacock")
            ((Peacock)var)->Fly();
    }
}
```

Конструкция несколько более громоздкая, но, если для каких-то целей необходимо имя класса, `typeid` является единственным способом получить его.

## Конструктор копирования

Среди методов, обеспечивающих работу с классами, особое место занимают конструкторы. Первая и самая важная их функция – это создание объекта перед началом работы с ним. Поэтому класс должен иметь хотя бы один конструктор.

На самом деле, конструкторов обычно бывает несколько:

1. Конструктор по умолчанию. Это конструктор без параметров, который задает некоторые начальные значения для полей класса так, чтобы с объектом, созданным этим конструктором, можно было начать работу.

2. Конструктор с параметрами. Такой конструктор позволяет создавать объекты с необходимым набором значений атрибутов для каждого объекта.

3. Конструктор копирования. Очень важный конструктор, без которого невозможна нормальная работа с объектами. Когда разработчик использует фундаментальные типы данных (целые, вещественные и т.д.), то операция присваивания понимается и выполняется однозначно – в переменную слева от знака `=` копируется значение справа от `=`.

```
int a,b;
a = 5;
b = a;
```

Переменная `b` получит значение 5, если будем изменять `a`, то на `b` это не отразится. Когда же идет работа с указателями, всё становится сложнее:

```
int *a,*b;
a = new int;
*a = 5;
b = a;
```

Переменная `b` будет указывать на ту же область памяти, что и `a`, и если будем изменять `*a`, то `*b` так же будет изменять значение. Мы получили не два разных объекта, а один с двумя именами. Если написать

```
*b = *a;
```

то ошибка будет исправлена. Но такой простой способ не подходит для классов, так как атрибуты класса в свою очередь могут быть объектами других классов, созданными с помощью указателей. В этом случае разименование указателя всё равно приведет к ошибке, а к тому же при присваивании происходит между объектами в целом, без использования информации о внутренней структуре объекта. Предположим, что фирму `f2` из предыдущего раздела решили создать как полную копию `f1`, по сути сделать своеобразный клон (часто используемый прием, когда исходный объект "портить" нельзя, а проверить какие-то задачи надо):

```
f2 = f1;
```

Операция будет выполнена, а вот результат будет несколько неожиданным – на самом деле списки отделов и списки сотрудников в отделах будут общими и выполнение инструкции

```
f2->RemoveEmpl ("Иванов");
```

приведет к увольнению директора из f1 тоже. Это произошло из-за того, что списки хранят указатели на объекты-сотрудники, о чем программист, использующий класс фирма, знать в принципе не обязан, так как реализация для него скрыта. Если разработчик хочет обеспечить надежную работу со всеми объектами в своем проекте, то конструктор копирования должен быть создан для каждого класса в проекте. Общий вид конструктора копирования следующий:

```
ИмяКласса :: ИмяКласса (ИмяКласса& obj) АтрибутКласса1 = obj.АтрибутКласса1;
// перечисление всех атрибутов класса АтрибутКласса2 = obj.АтрибутКласса2;
// для атрибутов-ссылок . заменяется на -> АтрибутКласса3 = obj->АтрибутКласса3;
...
```

В этом случае каждая операция присваивания будет выполняться следующим образом: если то, что стоит справа от оператора присваивания, обладает своим конструктором копирования, то будет вызван этот конструктор, если конструктора нет, то присваивание будет идти по обычным правилам языка.

```
Employer :: Employer (Employer& emp){
    name = emp.name;
    position = emp.position;
}
Department :: Department (Department& dep){
    name = dep.name;
    foreach (Employer emp in dep.lstEmployer)
        lstEmployer.push_back (emp);
}
Firma :: Firma (Firma& fm){
    name = fm.name;
    foreach (Department dep in fm.lstDepartment)
        lstDepartment.push_back (fm.dep);
}
```

Мы получим иерархию вызовов конструкторов копирования, и в этом случае присваивание

```
f2 = f1;
```

будет выполнено правильно.

## Использование паттернов. Пример использования паттерна Фабрика

**Определение 14. Паттерны** — это проверенные и готовые к использованию решения часто возникающих в повседневном программировании задач. Это не класс и не библиотека, которую можно подключить к проекту, это нечто большее. Кроме того, паттерны не зависят от языка программирования, и легко реализуются в большинстве языков.

Паттерны подразделяются на архитектурные паттерны, **паттерны проектирования**, паттерны анализа, паттерны тестирования.

В свою очередь, паттерны проектирования делятся на основные, **порождающие**, структурные, поведенческие, паттерны параллельного программирования, MVC, паттерны корпоративных систем.

**Определение 15. Порождающие (creational) паттерны** – определяют способы создания объектов в системе. Абстрагируют процесс инстанцирования, позволяют сделать систему независимой от способа создания, композиции и представления объектов. Отвечают на вопросы «кто, когда и как создаёт объекты в системе»

### Паттерн Abstract Factory (Абстрактная фабрика)

#### Название паттерна

- Abstract Factory / Абстрактная фабрика
- Toolkit / Инструментарий
- Factory / Фабрика

#### Цель паттерна

– предоставить интерфейс для проектирования и реализации семейства взаимосвязанных и взаимозависимых объектов, не указывая конкретных классов, объекты которых будут создаваться.

#### Пояснение

В соответствии с принципом инверсии зависимости (DIP) следует

- использовать зависимости от абстрактных классов и
- избегать зависимостей от конкретных классов, особенно когда последние изменчивы.

**Пример.** Нарушение принципа DIP:

```
Circle c = new Circle(origin, 1);
```

Здесь Circle – конкретный класс, значит, модули, которые создают экземпляры Circle, обязательно нарушают DIP.

1. **Любой код, в котором используется ключевое слово new, не согласуется с принципом DIP**
2. Часто нарушение принципа DIP практически безвредно

3. Чем выше вероятность того, что конкретный класс будет изменяться, тем вероятнее, что зависимость от него приведет к неприятностям
4. Если конкретный класс не склонен к изменениям, то ничего страшного в зависимости от него нет

**Пример.** Создание объектов типа `string` не вызывает беспокойства. Зависимость от класса `string` вполне безопасна, так как в обозримом будущем этот класс не изменится

## Когда следует использовать паттерн Abstract Factory

- система не должна зависеть от того, как в ней создаются и компонируются объекты;
- объекты, входящие в семейство, должны использоваться вместе;
- система должна конфигурироваться одним из семейств объектов;
- надо предоставить интерфейс библиотеки, не раскрывая её внутреннюю реализацию

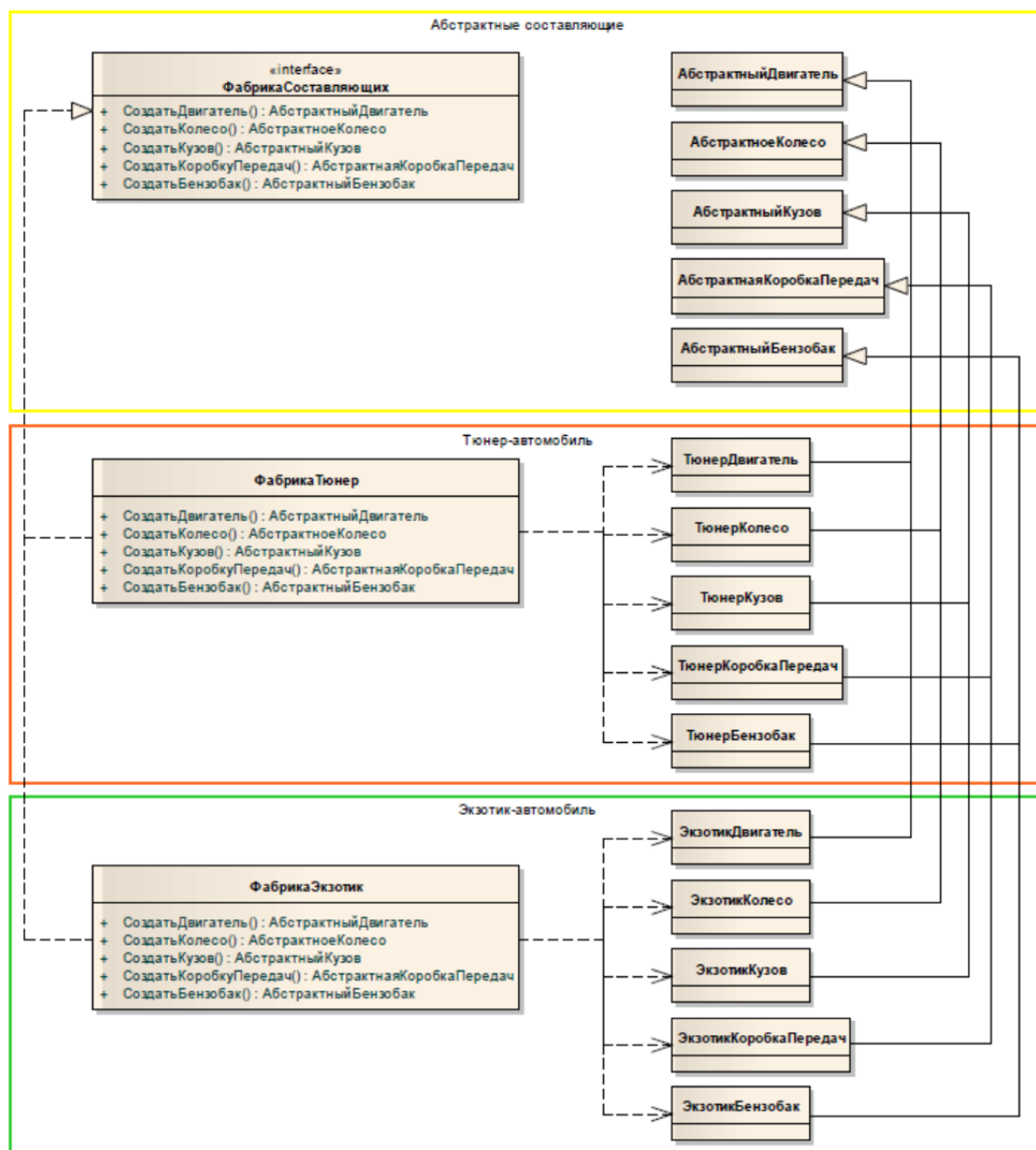
### Пример – игра «Супер Ралли» (гонка на автомобилях)

- Одно из требований: игрок должен иметь возможность **выбирать себе автомобиль для участия в гонках.**
- Каждый из автомобилей состоит из специфического набора составляющих:
  - двигатель, колеса, кузов, коробка передач, бензобак
  - определяют возможности автомобиля (скорость, маневренность, устойчивость к повреждениям, длительность непрерывной гонки без перезаправки и д.р.).
- Может быть много разных типов автомобилей.
- Их количество может изменяться динамически (например, в зависимости от опыта игрока).
- Клиентский код, выполняющий конфигурирование автомобиля специфичным семейством составляющих, **не должен зависеть от типа выбранного автомобиля.**

### Предлагаемая реализация

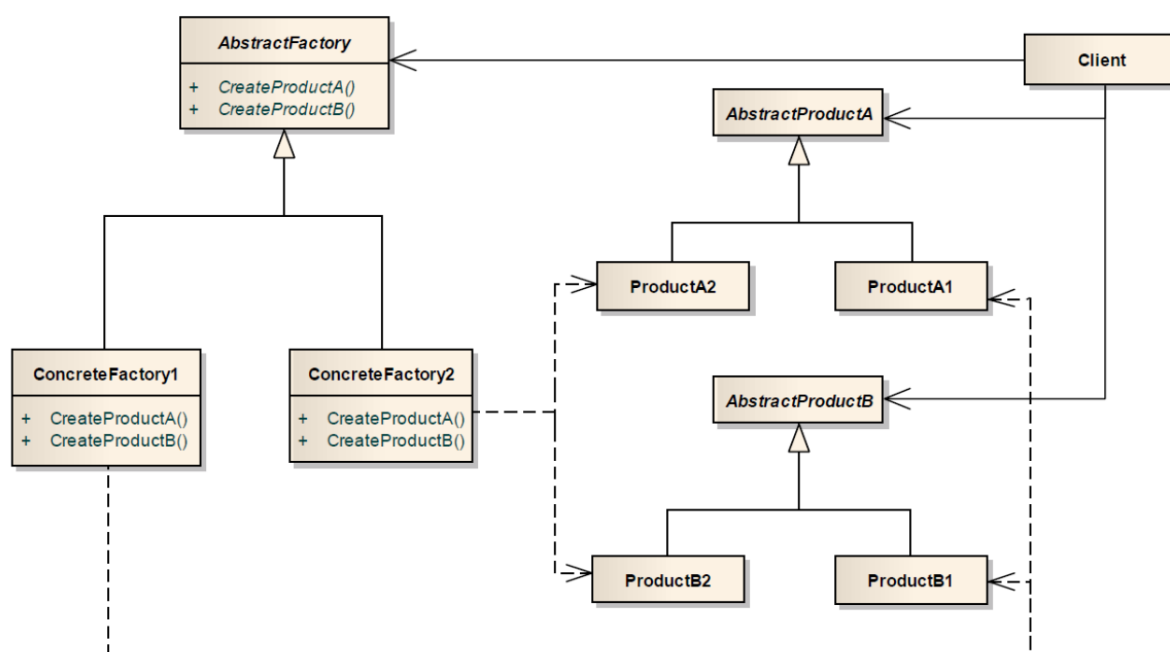
- Создается интерфейс `ФабрикаСоставляющих` – предназначен для создания конкретных классов (фабрик), которые будут создавать семейства составляющих для каждого конкретного типа автомобиля.
- Методы этого класса должны возвращать ссылки на абстрактные составляющие, что позволит в конкретных классах-фабриках, создавать конкретные составляющие (подклассы абстрактных составляющих).

## Диаграмма классов



- Клиентский код, который «собирает» автомобиль из деталей, использует интерфейсную ссылку **ФабрикаСоставляющих**,
  - методы данного интерфейса возвращают ссылки на абстрактные составляющие.
- Можно передавать клиенту объект конкретной фабрики, которая создает семейство объектов конкретных составляющих

## Общая структура паттерна Abstract Factory



## Участники паттерна Abstract Factory

- Интерфейс **AbstractFactory** — абстрактная фабрика
  - Предоставляет общий интерфейс для создания семейства продуктов.
- Класс **ConcreteFactory** — конкретная фабрика
  - Реализует интерфейс AbstractFactory и создает семейство конкретных продуктов.
- Метод интерфейса **AbstractProduct** — абстрактный продукт
  - Предоставляет интерфейс абстрактного продукта, ссылку на который возвращают методы фабрик.
- Метод класса **ConcreteProduct** — конкретный продукт
  - Реализует конкретный тип продукта, который создается конкретной фабрикой.

## Отношения между участниками

- Клиент знает только о существовании абстрактной фабрики и абстрактных продуктов.
- Для создания семейства конкретных продуктов клиент конфигурируется соответствующим экземпляром конкретной фабрики.
- Методы конкретной фабрики создают экземпляры конкретных продуктов, возвращая их в виде ссылок на соответствующие абстрактные продукты.

## Достоинства использования паттерна

- Позволяет изолировать конкретные классы продуктов.
- Клиент знает о существовании только абстрактных продуктов, что ведет к упрощению его архитектуры.

- Упрощает замену семейств продуктов — для использования другого семейства продуктов достаточно конфигурировать клиентский код соответствующей конкретной фабрикой.
- Дает гарантию сочетаемости продуктов — так как каждая конкретная фабрика создает группу продуктов, то она и следит за обеспечением их сочетаемости.

## Недостатки использования паттерна

- Трудно поддерживать новые виды продуктов, которые содержат другой состав компонент.
- Для добавления нового продукта необходимо изменять всю иерархию фабрик, а также клиентский код.

## Практический пример использования паттерна

- Задача – разработать ПО для магазина компьютерной техники.
- Одно из требований – быстрое создание конфигурации системного блока.
- Предположим, что в состав конфигурации системного блока входят:
  1. бокс (Box);
  2. процессор (Processor);
  3. системная плата (MainBoard);
  4. жесткий диск (Hdd);
  5. оперативная память (Memory).
- Допустим, что программа должна создавать шаблоны типичных конфигураций двух типов:
  - домашняя конфигурация;
  - офисная конфигурация.
- Для всех этих конфигураций определим абстрактный класс.
- Конкретные модели составляющих будем определять путем наследования от абстрактного базового класса

## Класс персонального компьютера Pc

- Класс, представляющий конфигурацию системного блока:

```
public class Pc
{
    public Box Box { get; set; }
    public Processor Processor { get; set; }
    public MainBoard MainBoard { get; set; }
    public Hdd Hdd { get; set; }
    public Memory Memory { get; set; }
}
```

## Интерфейс фабрики создания конфигурации системного блока

- Ответственность за их создание заданной конфигурации надо возложить на один класс-фабрику.

- Эта фабрика должна реализовать интерфейс **IPcFactory**.
- Методы это интерфейса возвращают ссылки на классы абстрактных продуктов.

```
public interface IPcFactory
{
    Box CreateBox ( ) ;
    Processor CreateProcessor ( ) ;
    MainBoard CreateMainBoard ( ) ;
    Hdd CreateHdd ( ) ;
    Memory CreateMemory ( ) ;
}
```

- Для создания компонентов конфигураций определяем классы конкретных фабрик
  - **HomePcFactory**
  - **OfficePcFactory**
- В каждом из **create**-методов этих классов создается объект конкретного класса продукта, соответствующего типу конфигурации.

## Класс HomePcFactory

- Фабрика для создания "домашней" конфигурации системного блока ПК

```
public class HomePcFactory : IPcFactory
{
    public Box CreateBox()
        { return new SilverBox(); }
    public Processor CreateProcessor()
        {return new IntelProcessor(); }
    public MainBoard CreateMainBoard()
        { return new MSIMainBord(); }
    public Hdd CreateHdd()
        { return new SamsungHDD(); }
    public Memory CreateMemory()
        { return new Ddr2Memory();}
}
```

## Класс OfficePcFactory

- Фабрика для создания "офисной" конфигурации системного блока ПК

```
public class OfficePcFactory : IPcFactory
{
    public Box CreateBox()
        { return new BlackBox(); }
    public Processor CreateProcessor()
        { return new AmdProcessor();}
    public MainBoard CreateMainBoard()
        {return new AsusMainBord(); }
    public Hdd CreateHdd()
```

```

        {return new LGHDD();}
    public Memory CreateMemory()
        { return new DdrMemory(); }
}

```

## Класс PcConfigurator

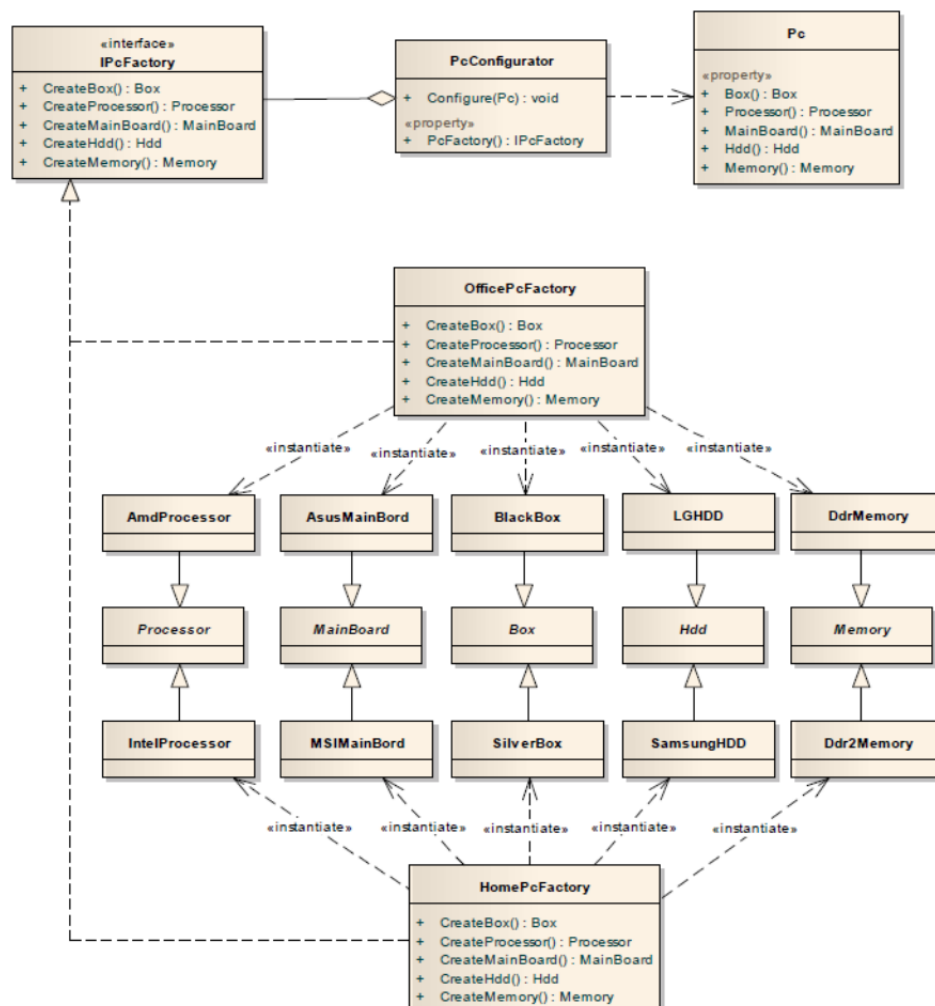
- Ответственен за создание объекта типа Pc выбранного типа

```

public class PcConfigurator {
    public IPcFactory PcFactory { get; set; }
    public void Configure(Pc pc) {
        pc.Box = PcFactory.CreateBox();
        pc.MainBoard = PcFactory.CreateMainBoard();
        pc.Hdd = PcFactory.CreateHdd();
        pc.Memory = PcFactory.CreateMemory();
        pc.Processor = PcFactory.CreateProcessor();
    }
}

```

## Полная диаграмма классов



- Класс **PcConfigurator** принимает экземпляр конкретной фабрики и с помощью её методов создает составляющие персонального компьютера.
- **PcConfigurator** работает с интерфейсной ссылкой **IPcFactory** и ничего не знает о конкретных фабриках конфигураций и конкретных составляющих.
- Сила абстрактной фабрики
  - конкретную фабрику можно определять на этапе выполнения программы
  - клиентский код не зависит от конкретных фабрик или конкретных продуктов

## Использование паттернов. Пример использования паттерна Одиночка

### Паттерны Singleton (Одиночка) и Monostate (Моносостояние)

• Обычно между классами и их объектами существует отношение один-ко-многим:

1. Можно создавать много экземпляров одного класса
2. Объекты создаются, когда они требуются, и уничтожаются, когда перестают быть необходимыми

• Однако у некоторых классов должен быть только один объект:

1. Создается в начале работы программы
2. Уничтожается при ее завершении
3. Такие объекты могут быть:
  - **корневыми объектами приложения**, – из которых можно добраться до других объектов системы.
  - **фабриками**, порождающими другие объекты.
  - **менеджерами**, которые следят за другими объектами.
4. Во всех случаях требуется только один такой объект
5. Наличие нескольких таких объектов – серьезная логическая ошибка:
6. — Если есть *более одного корня*, то доступ к объектам приложения может зависеть от выбранного корня. Программисты, не знающие о наличии нескольких корней, могут, не сознавая того, видеть лишь подмножество всех объектов приложения.
  - Если существует *несколько фабрик*, то может быть потерян контроль над созданными объектами.
  - При наличии *нескольких менеджеров* операции, предполагавшиеся последовательными, могут оказаться параллельными.

Может показаться, что вводить специальные механизмы обеспечения единственности таких объектов – излишество. В конце концов, на этапе инициализации приложения можно просто создать по одному экземпляру каждого, и дело с концом.

Такой шаблон иногда называется «Просто создай одного».

Отметим, что различие между двумя описанными паттернами – это различие между поведением и структурой. Паттерн Одиночка навязывает структуру единственности, не позволяя создать более одного экземпляра. Моносостояние, напротив, навязывает поведение единственности, не налагая структурных ограничений.

Это различие станет понятным, если заметить, что тесты для паттерна Моносостояние проходят и для класса Singleton, однако у класса Monostate нет ни малейшей надежды пройти тесты для Одиночки.

## Паттерн Одиночка (Singleton)

- Одиночка (Singleton) – очень простой паттерн.

```
public class Singleton {
    private static Singleton theInstance = null;
    private Singleton() {}
    public static Singleton Instance {
        get {
            if (theInstance == null)
                theInstance = new Singleton();
            theInstance;
        }
    }
}
```

## Пример класса

- Описание класса

```
public class Singleton {
    private static Singleton instance;
    private Singleton() { }
    public static Singleton Instance {
        get {
            if (instance == null)
                {instance = new Singleton(); }
            return instance;
        }
    }
    public String DataItem { get; set; }
}
```

- Использование класса

```
Singleton single = Singleton.Instance;
single.DataItem = "value";
```

## Достоинства

- Применимость к любому классу.
  - Любой класс можно преобразовать в Одиночку, если сделать его конструкторы закрытыми и добавить соответствующие статические методы и переменную-член.
- Может быть создан путем наследования.
  - Имея некоторый класс, можно создать его подкласс, который будет Одиночкой.
- Отложенное вычисление.
  - Если Одиночка не используется, то он и не создается.

## Недостатки

- Уничтожение не определено.
  - Не существует приемлемого способа уничтожить или «списать» Одиночку.
  - Даже если добавить метод, обнуляющий переменную `theInstance`, другие модули могут хранить у себя ссылку на Одиночку.
  - При последующих обращениях к `Instance` будет создан новый экземпляр, что приведет к образованию двух одновременно существующих экземпляров.
  - Эта проблема особенно остро стоит в языке C++, где экземпляр может быть уничтожен, что приведет к разыменованию уже не существующего объекта.
- Не наследуется.
  - Класс, производный от Одиночки, сам не является Одиночкой.
  - Если необходимо, чтобы он был Одиночкой, придется добавить статический метод и переменную-член.
- Эффективность.
  - Каждое обращение к свойству `Instance` приводит к выполнению предложения `if`.
  - Для большинства обращений это предложение бесполезно.
- Непрозрачность.
  - Пользователи Одиночки знают, с чем имеют дело, потому что вынуждены обращаться к свойству `Instance`.

## Пример использования шаблона Одиночка

- Предположим, что имеется веб-приложение, позволяющее пользователям входить в защищенные области сервера.
- В таком приложении будет какая-то БД, которая содержит описания пользователей:
  - имена,
  - пароли
  - другие атрибуты.
- Доступ к базе данных выполняется с помощью специального API.

## Варианты реализации

- Можно было бы в каждом модуле, которому необходимо читать и изменять данные о пользователях, обращаться к базе напрямую.
- Недостаток:
  - вызовы стороннего API будут разбросаны по всему коду,
  - это затрудняет следованию соглашения о доступе и структуре программы.
- Лучше воспользоваться паттерном **Facade (Фасад)** и создать класс **UserDatabase**, предоставляющий методы для чтения и изменения объектов **User**.
  - Это частный случай шаблона Фасад, называемый шаблоном Шлюз (Gateway).
- Методы обращаются к стороннему API доступа к базе данных, осуществляя отображение между объектами **User** и таблицами базы.
- Внутри класса **UserDatabase** можно обеспечить соглашения о структуре и порядке доступа.
  - Например, можно гарантировать, что не будет добавлена запись **User**, в которой поле **username** пусто.

– Или упорядочить обращения к записи **User**, так, чтобы никакие два модуля не могли одновременно читать и изменять ее.

## Решение на основе паттерна Singleton (Одиночка)

- Создается класс с именем **UserDatabaseSource**, который реализует интерфейс **UserDatabase**.
- В коде свойства **Instance** нет традиционного предложения **if**, защищающего от многократного создания.
- Вместо этого используется механизм статической инициализации, имеющийся в .NET.

## Интерфейс **UserDatabase** и класс **UserDatabaseSource**

```
public interface UserDatabase {
    User ReadUser(string userName);
    void WriteUser(User user);
}

public class UserDatabaseSource : UserDatabase {
    private static UserDatabase theInstance = new
        UserDatabaseSource();
    public static UserDatabase Instance {
        get { return theInstance; }
    }
    private UserDatabaseSource() { }
    public User ReadUser(string userName) {
        // Реализация
    }
    public void WriteUser(User user) {
        // Реализация
    }
}
```

- Такое использование паттерна Одиночка распространено чрезвычайно широко.
- Гарантируется, что весь доступ к базе данных производится через единственный экземпляр **UserDatabaseSource**.
- При этом в **UserDatabaseSource** очень легко вставлять
  - различные проверки,
  - счетчики,
  - блокировки,
 которые обеспечивают выполнение требуемых соглашений о порядке доступа и структуре кода.

# Использование паттернов. Пример использования паттерна Фабричный метод

## Название и классификация

Фабричный метод – паттерн, порождающий объекты

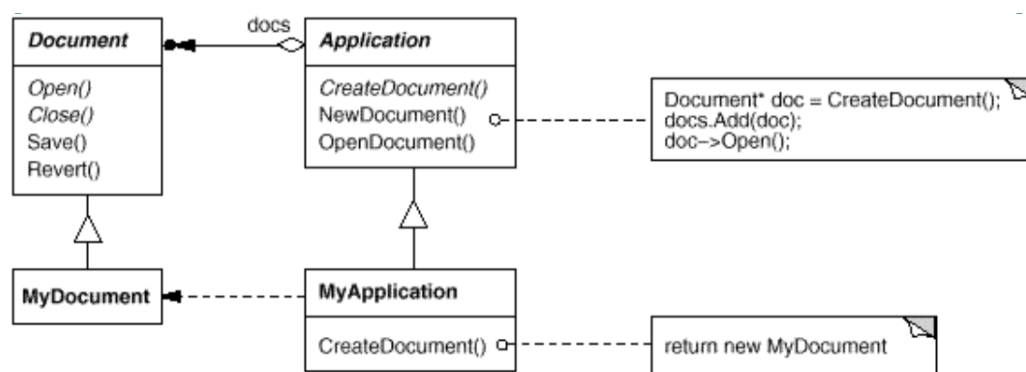
## Назначение

Определяет интерфейс для создания объекта, но оставляет подклассам решение о том, какой класс инстанцировать. Фабричный метод позволяет классу делегировать инстанцирование в подклассы

## Известен также под именем

Virtual Constructor

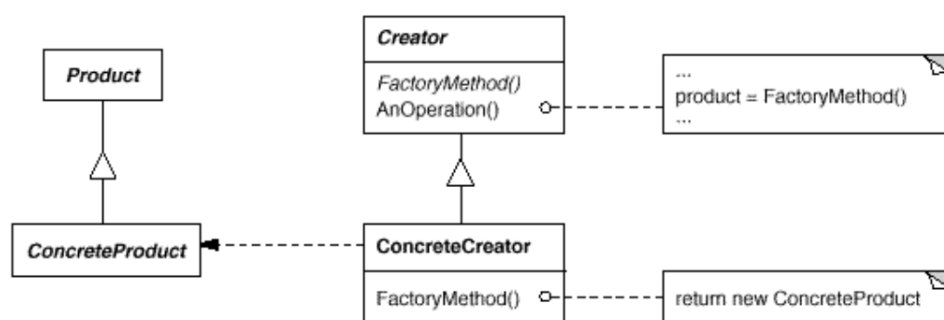
## Мотивация



## Применимость

Используйте паттерн фабричный метод, когда:

- классу заранее неизвестно, объекты каких классов ему нужно создавать
- класс спроектирован так, чтобы объекты, которые он создает, специфицировались подклассами
- класс делегирует свои обязанности одному из нескольких вспомогательных подклассов, и вы планируете локализовать знание о том, какой класс принимает эти обязанности на себя



## Особенности

- Две основных разновидности:
  1. Класс `Creator` – является абстрактным и не содержит реализации объявленного в нем фабричного метода.
  2. `Creator` – конкретный класс, в котором по умолчанию есть реализация фабричного метода
- Параметризованные фабричные методы
  - Это еще один вариант паттерна, который позволяет фабричному методу создавать разные виды продуктов. Фабричному методу передается параметр, который идентифицирует вид создаваемого объекта.

## Пример на C sharp

Теперь рассмотрим на реальном примере. Допустим, мы создаем программу для сферы строительства. Возможно, вначале мы захотим построить многоэтажный панельный дом. И для этого выбирается соответствующий подрядчик, который возводит каменные дома. Затем нам захочется построить деревянный дом и для этого также надо будет выбрать нужного подрядчика:

```
class Program
{
    static void Main(string[] args)
    {
        Developer dev = new PanelDeveloper("ООО КирпичСтрой");
        House house2 = dev.Create();

        dev = new WoodDeveloper("Частный застройщик");
        House house = dev.Create();

        Console.ReadLine();
    }
}
// абстрактный класс строительной компании
abstract class Developer
{
    public string Name { get; set; }

    public Developer (string n)
    {
        Name = n;
    }
    // фабричный метод
    abstract public House Create();
}
// строит панельные дома
class PanelDeveloper : Developer
{
    public PanelDeveloper(string n) : base(n)
    { }
}
```

```

        public override House Create()
        {
            return new PanelHouse();
        }
    }
    // строит деревянные дома
    class WoodDeveloper : Developer
    {
        public WoodDeveloper(string n) : base(n)
        { }

        public override House Create()
        {
            return new WoodHouse();
        }
    }

    abstract class House
    { }

    class PanelHouse : House
    {
        public PanelHouse()
        {
            Console.WriteLine("Панельный дом построен");
        }
    }
    class WoodHouse : House
    {
        public WoodHouse()
        {
            Console.WriteLine("Деревянный дом построен");
        }
    }
}

```

В качестве абстрактного класса Product здесь выступает класс House. Его две конкретные реализации - PanelHouse и WoodHouse представляют типы домов, которые будут строить подрядчики.

В качестве абстрактного класса создателя выступает Developer, определяющий абстрактный метод Create(). Этот метод реализуется в классах-наследниках WoodDeveloper и PanelDeveloper. И если в будущем нам потребуется построить дома какого-то другого типа, например, кирпичные, то мы можем с легкостью создать новый класс кирпичных домов, унаследованный от House, и определить класс соответствующего подрядчика.

Таким образом, система получится легко расширяемой. Правда, недостатки паттерна тоже очевидны - для каждого нового продукта необходимо создавать свой класс создателя.

## **Понятие жизненного цикла разработки программного обеспечения. Виды моделей ЖЦ**

.

## Основные принципы создания хорошего кода

**Технологии COM, CORBA, .NET. Их  
идеология и использование. Автоматное  
программирование**

## Понятие тестирования. Основные типы тестирования